UNIVERSITY OF MILANO-BICOCCA

Department of Computer Science, Systems and Communication

Master's Degree Programme in **Computer Science**

Innovative Management of Service Requests: From a Distributed Monolith to a Microservices Architecture, with RAG-Enhanced Reccomendation System

Advisor: Prof. Leonardo Mariani

Master's Thesis by:
Nicolas Guarini
Student ID: 918670

Academic Year 2024–2025

"What I cannot create, I do not understand"

- Richard Feynman

Contents

Introduction

The technological evolution of recent decades has profoundly transformed the business landscape, making IT solutions a crucial factor for the success of enterprises. In this context, Elmec Informatica S.p.A. stands out as a significant player in the Information Technology sector in Italy. Founded in 1971 and headquartered in Varese, Elmec combines extensive industry experience with a strong focus on innovation, offering services and solutions that cover a wide range of business needs, offering advanced IT solutions for medium and large enterprises.

The company manages four datacenters (including one Tier IV certified), provides Hybrid IT services, digital workspace and Device-as-a-Service solutions, and is a key partner for cybersecurity and regulatory compliance. With over 450 certified technicians, Elmec integrates cloud technologies (in collaboration with AWS and GAIA-X), promotes environmental sustainability, and stands out for its innovation through its Technological Campus and ongoing investments in professional training.

The primary context of this internship revolves around the redesign and enhancement of the existing *Service Request Portal (SRP)*, which is essential for managing client service requests efficiently.

A Service Requests is a formal request submitted by a customer to obtain specific services or actions within their IT infrastructure. These requests can encompass a wide range of activities, from creating new user accounts in the Active Directory system to managing user permissions, resetting passwords, and deploying software updates.

Various challenges have been identified within the current architecture, particularly concerning the dynamic forms used by customers for submitting the service requests, the management of customizable fields, and the integration with external data sources.

The main objectives of this internship are:

- **Analysis of the Existing System**: Conduct a comprehensive analysis of the current state of the SRP system, identifying weaknesses and inefficiencies that hinder its performance and user experience;
- **Architectural redesign**: Propose and implement a complete architectural redesign of the system;

- **Modular and Scalable Solution**: Design a modular and scalable solution that can accomodate various client-specific configurations;
- **Migrations**: Plan and execute a full migration of databases and other company services/platforms that use SRP.
- **Fine-tuned LLMs integration**: Explore opportunities for integrating specifically fine-tuned LLM systems into the SR workflow.

Chapter 1

Overview of the existing System

The internship project was not focused on creating an entirely new system from scratch, but rather on a deep redesign and restructuring of an existing system developed many years ago using legacy technologies and affected by several issues and critical limitations.

Before discussing the architectural, technological, and implementation choices that were made, it is necessary to carry out a thorough analysis of the current system, in order to understand its behavior, operational flows, technologies used, the main issues to be addressed, how to resolve them, how to prevent them from reoccurring in the future, and how to ensure that the new system is scalable and maintainable.

1.1 Service Requests Workflow

The lifecycle of a Service Request within the current system is orchestrated through a series of defined states and automated processes, involving interactions between various components and human operators. The flow is designed to manage SRs from initiation to completion, handling approvals, execution, and potential errors [19].

In Figure 1.1 [19], a simple and high-level flowchart is shown that illustrates the main flows and scenarios.

1.1.1 SR Initiation and Initial State Management

A Service Request can be initiated through two primary channels [19]:

- **Via MyElmec Portal**: Clients can submit an SR by completing a dedicated form from the MyElmec portal. Upon saving the form, a ticket is created in the HelpdeskAdvanced (HDA) system with a `QUALIFIED` status, and a corresponding Service Request is generated in SRP with a `NEW` status. These two entities are then linked via the HDA Ticket ID;
- **Via HDA Ticket by Elmec Operator**: Operators have the ability to associate a catalog SR directly with a ticket they are managing within the HDA interface. When the ticket is saved in a `DELEGATED` status, the SR

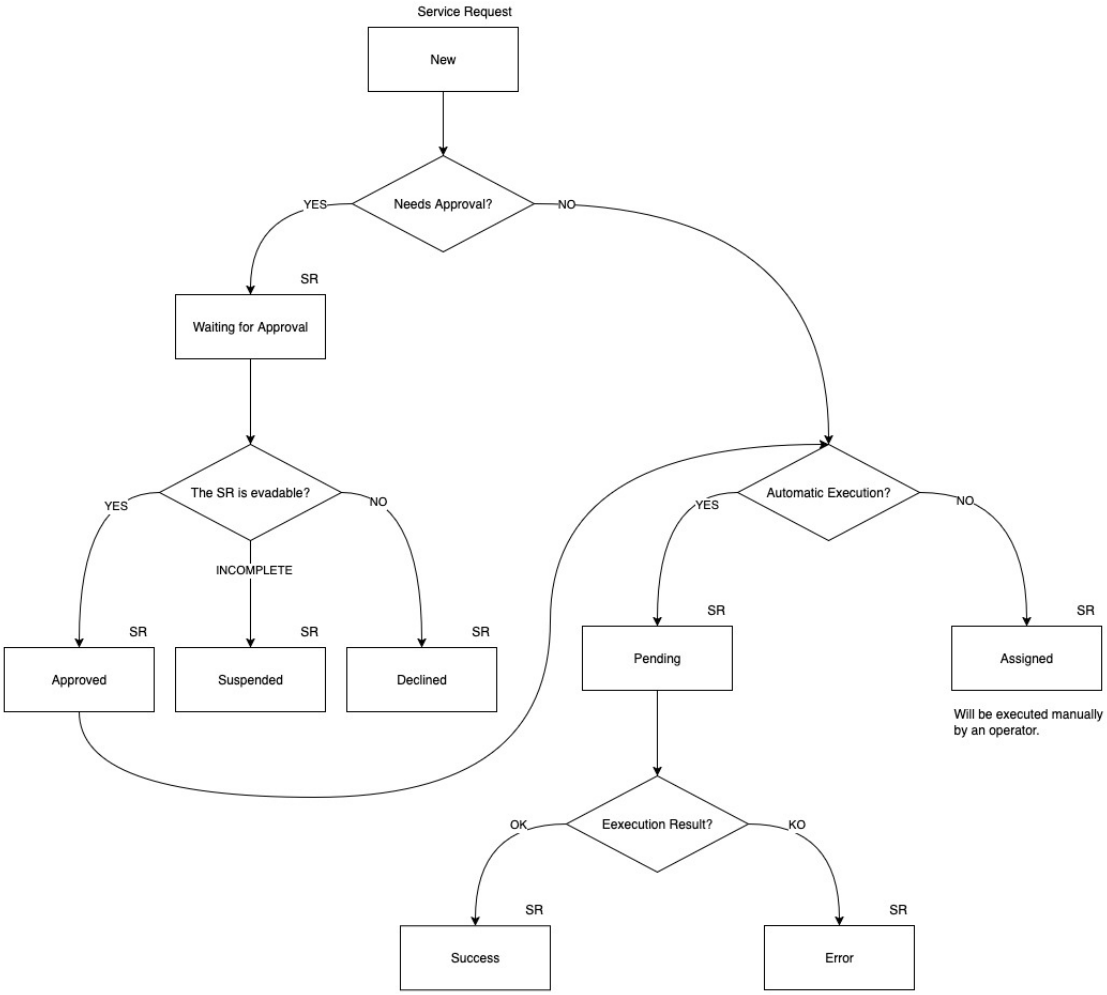

Figure 1.1: High-Level Service Request Lifecycle

is created in SRP with a `NEW` status, linked to the ticket, and immediately triggers the SR workflow within SRP.l

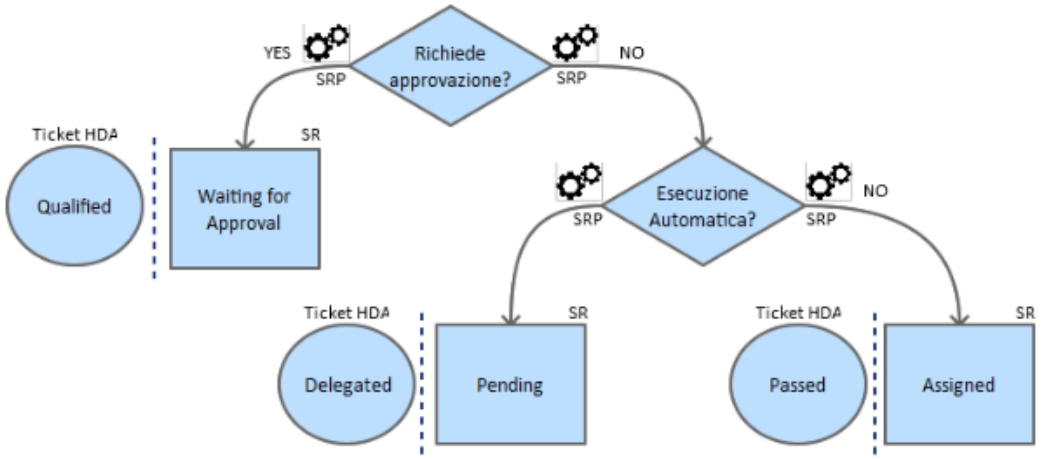

Figure 1.2: AS-IS New Service Request Workflow

As shown in Figure 1.2, following the Service Request creation, the system evaluates whether the Service Request requires approval by the operators. The logic for setting the initial SR and HDA ticket states is as follows [19]:

- **If approval is required**: the Service Request is set to `WAITING FOR APPROVAL` status, and the corresponding HDA ticket is set to `QUALIFIED`. A communication is added to the ticket, indicating that the SR needs validation from an operator before execution;
- **If no approval is required and it's linked to an automatism**: the Service Request enters a `PENDING` state, and the HDA ticket is set to `DELEGATED`;
- **If no approval is required and it's not linked to an automatism**: the Service Request is `ASSIGNED`, and the HDA ticket is set to `PASSED`. A communication is added to the ticket, noting that the SR requires manual processing by an operator.

1.1.2 Approval and Execution Workflows

As shown in Figure 1.3, for SRs requiring approval (`WAITING FOR APPROVAL` state with an HDA `QUALIFIED` ticket), an operator's intervention is necessary to validate the request. The operator has three possible actions [19]:

- **Cancel the SR**: the HDA ticket is set to `CLOSED ABORTED`, the SR status becomes `DECLINED`, and a notification email is sent to the requesting client;
- **Request client modification**: the HDA ticket is set to `WAITING USER`, the SR status becomes `SUSPENDED`, and a notification email is sent to

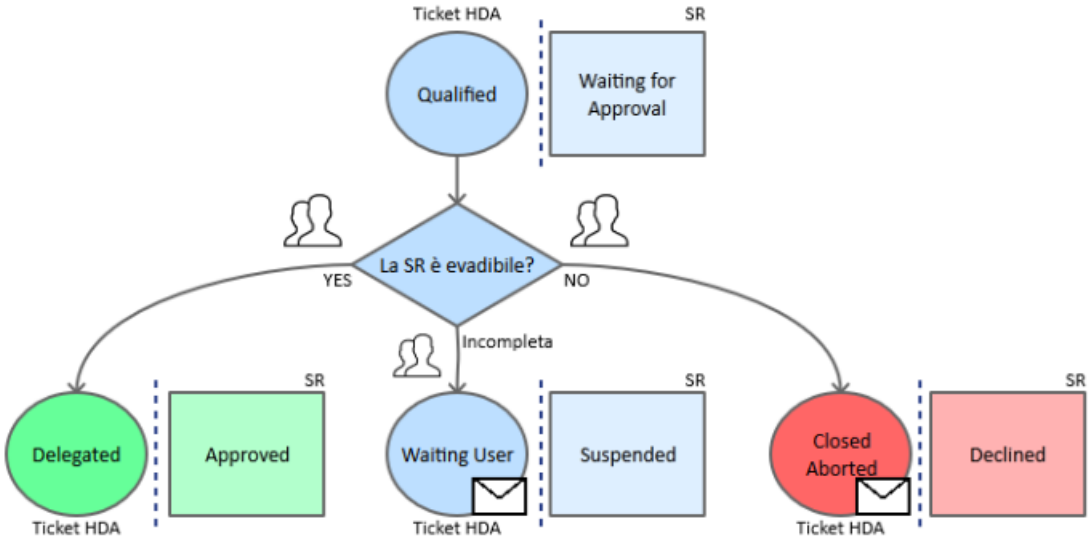

Figure 1.3: AS-IS Approval Service Request Workflow

the client. The workflow pauses until the client modifies the SR via Elmec.com;

- **Approve the SR**: the HDA ticket is set to `DELEGATED`, and the SR status becomes `APPROVED`.

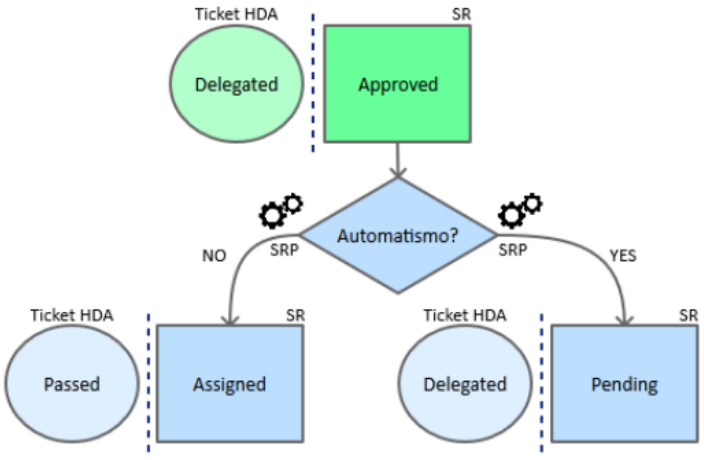

Figure 1.4: AS-IS Approved Service Request Workflow

As shown in Figure 1.4, once a SR is approved, SRP manages its subsequent flow based on whether it is linked to an automatism [19]:

- **If the SR is linked to an automatism**: the SR transitions to a `PENDING` state, while the HDA ticket remains `DELEGATED`. This implies it's awaiting automated execution;

- **If the SR is not linked to an automatism**: the SR is immediately set to `ASSIGNED`, and the HDA ticket is moved to `PASSED`. A note is added

to the ticket indicating that the SR requires manual processing.

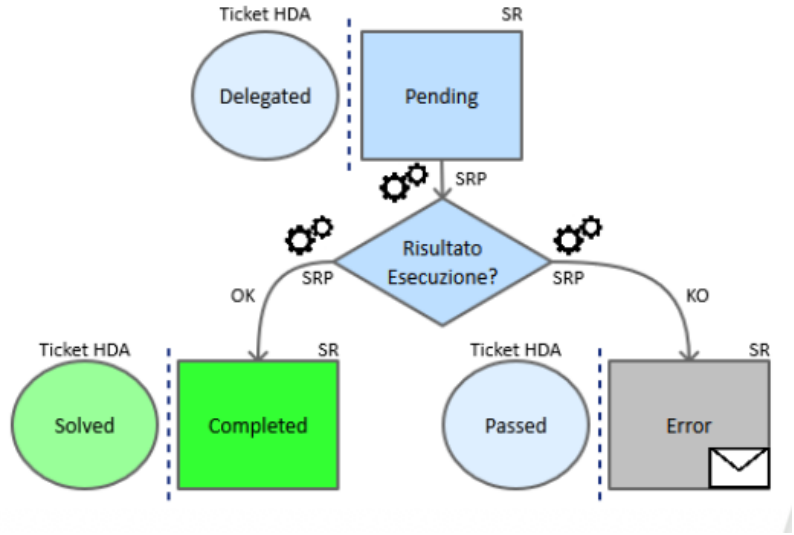

Figure 1.5: AS-IS Pending Service Request Workflow

As shown in Figure 1.5, an SR in `PENDING` state is awaiting the execution of its associated automatism. The outcome of this automated execution determines the next state [27]:

- **Successful execution**: the SR is set to `COMPLETED`, and the corresponding HDA ticket is set to `SOLVED`;
- **Unsuccessful execution**: the SR transitions to an `ERROR` state, and the HDA ticket is set to `PASSED`. Communication regarding the error is added to the ticket for visibility within HDA.

The execution of Service Requests themselves can involve the SRP Agent on the client's Domain Controller for operations requiring client-side interaction (e.g., Active Directory or Exchange environment changes). The SRP Agent periodically checks `ELMEC-SRP01` for pending SRs. If found, it retrieves the SR details, loads the relevant PowerShell script, and executes it. The agent then informs ELMEC-SRP01 that the SR is running [27]. Monitoring is performed by a scheduled `READLOG` task on the client side, which checks the PowerShell script's log file every minute. A "KEY" in the log indicates completion, prompting the SRP Agent to inform ELMEC-SRP01 to set the SR to COMPLETED, triggering an HDA status update. An "ERROR" key indicates failure, leading to the log being sent to ELMEC-SRP01 and an error signal. A timeout mechanism also signals an error if completion is not detected within a specified duration [27].

A constraint that must be respected is that the high-level flow (state transitions and mapping between SR states and HDA ticket states) must not be modified, as it represents an established and approved process that also encompasses organizational and accountability aspects among various Elmec departments

[20]: those who configure and manage the catalog and related Service Requests (Managed Services (MxS) team), those who develop and maintain the product (Innovation team), and those responsible for the ticket management system (HDA team). The technological and implementation aspects, on the other hand, can and should be redesigned and improved [20], as will be explored in the following chapters.

1.2 Current System Architecture

The *SRP* system is fundamentally divided into two primary segments:

- Company Network;
- Client Network.

These segments interact and communicate exclusively through a *DMI Console* [21], which acts as a unidirectional bridge, transferring requests from the client network to the internal company network [21] [9].

The overall architecture is depicted in the diagram in figure 1.6 [21].

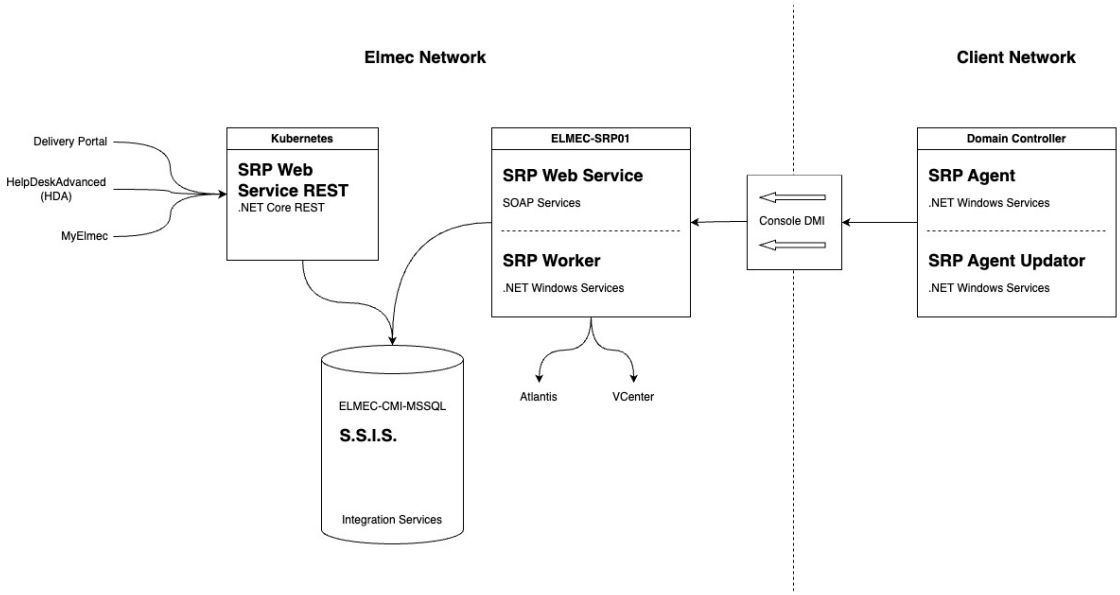

Figure 1.6: AS-IS System Architecture

The system's operational flow involves several key components across both networks, orchestrating the processing of service requests.

1.2.1 Main Functionalities and Components

The current *SRP* system relies on a suite of interconnected components, each serving a specific role in the lifecycle of a service request.

Elmec's Network Components:

- **SRP Web Sevice REST** [26]: This component, deployed on a Kubernetes cluster, exposes .NET Core RESTful services [21]. It serves as the primary interface for external systems such as:
  - **Delivery Portal**: An asset management system used to manage Service Request catalog configuration and Elmec's internal and client servers;
  - **HDA (HelpDeskAdvanced)**: A ticket management system. Each service request is intrinsically linked to an HDA ticket via an ID, and HDA is responsible for initiating and associating service requests with existing tickets.
  - **MyElmec**: A client-facing portal enabling customers to create, manage, and monitor the various services provided by Elmec (e.g., servers, management software, cloud solutions).

  The *SRP Web Service REST* communicates with the *ELMEC-CMI-MSSQL* server.
- **ELMEC-CMI-MSSQL**: This server hosts the central SQL Server Database for the SRP system. It is the persistent storage layer for all service request data and related information [21].
- **S.S.I.S. (SQL Server Integration Services)**: Running on the ELMEC-CMI-MSSQL server, SSIS packages are crucial for data transformation and import/export operations. They facilitate the transfer of data, such as master data (e.g., Active Directory users, Organizational Units, groups) sent by SRP Agents, into the database. Each SSIS package is designed to transform the received information for database insertion or vice versa [21].
- **ELMEC-SRP01**: This Windows Server functions as the core engine for service requests within the Elmec network. It houses all the PowerShell scripts associated with individual service requests, which are maintained by the Platform team [21]. Key components residing on ELMEC-SRP01 include:
  - **SRP WebServiceSoap**: This component provides SOAP services primarily for communication with the SRP Agent located on the client side. It interacts with the database to retrieve necessary information before exposing it to the agents for service request execution [25].
  - **SRP InternalWorker**: A Windows service that operates similarly to the SRP Agent but executes entirely within the Elmec network. This is utilized for service requests that do not require interaction with the client's network or Active Directory (e.g., "no patch" service

requests). The InternalWorker queries the database every 10 seconds for "internal worker" type jobs (like removing a server from a patching session, or resetting a managed Virtual Machine), connects to the SOAP service, retrieves the Service Request ID and its fields, and initiates job execution. It also monitors logs every 30 seconds to determine the job's status [24].

The only component in the client's network is called **Domain Controller**, which represents the client's Active Directory domain controller, where the communication tools with Elmec are installed. It hosts:

- **SRP Agent (.NET Windows Service)**: Typically, one agent is configured per client domain, specifically for Active Directory (AD) and Exchange-related service requests. This agent performs two main functions [27]:
  - Every 8 hours, it reads master data (AD users, OUs, groups) from the domain controller and sends it to ELMEC-SRP01 for storage via SSIS.
  - Every 5 minutes, it queries ELMEC-SRP01 for pending Service Requests. Upon finding any, it requests detailed information, loads the corresponding PowerShell script, and executes it. After initiating the script, the agent informs ELMEC-SRP01 that the SR is "running" and proceeds to the next SR.
- **SRP Agent Updator (.NET Windows Service)**: This service continuously monitors the SRP Agent folder for a file with a .new extension. If detected, it stops the SRP Agent, updates it, and restarts it

1.2.2 Interactions and Communication between Customers and Company networks

The Communication between the company network and the clients' networks is a critical aspect of the SRP system's operations and is handled through the **DMI Console**.

The DMI console serves as a key infrastructural component, primarily acting as a communication bridge between the customer's network and Elmec's internal network. Specifically, for the SRP system, the DMI console is responsible for transferring service requests (SR) from the customer's network to Elmec's internal network, although it does not handle traffic in the opposite direction [9].

Operation of the DMI Console

The connectivity of the DMI appliances to Elmec is established through multiple OpenVPN connections, initiated by the appliance towards the Brunello 4 and Mendrisio Data Centers (for Swiss customers). The DMI consoles are not directly exposed to the Internet and are accessible only by authorized personnel. Hardware requirements for a DMI node include 4 GB of RAM, 4 cores, and 50 GB of disk space [9].

Following recent developments in the DMI, the new consoles use K3S for container management. K3S is a lightweight, certified Kubernetes distribution designed for edge, IoT, and CI/CD environments. It is characterized by being a single binary that reduces external dependencies and uses SQLite as the default database, greatly simplifying installation and management. It already includes essential components such as *containerd* [13].

All consoles are centrally managed via Rancher, and the deployment of the application stack is delegated to ArgoCD, which ensures that container versions are constantly updated. For security reasons, the console in the customer network by default exposes only SSH, while services such as SRP are selectively enabled and only when necessary. All application data present on the consoles is stored on a LUKS (Linux Unified Key Setup) encrypted volume[1], which can only be unlocked if the VPN tunnel is established. The operating system update is performed through a standard Elmec system, called PMD (Patching Management Dashboard) [9].

1.3 Problems and core issues of the current system

The analysis of the architecture and workflow of the current SRP system has revealed a number of critical issues that compromise its scalability, maintainability, and reliability. These problems are deeply rooted in the technological and design choices made years ago and are evident across several key areas of the system, from catalog management to script execution and the import of master data from Active Directory.

[1]A platform-independent hard disk encryption method that can be used to encrypt disks across a wide variety of tools. This not only ensures full compatibility and interoperability between different software but also guarantees that password management occurs in a secure and documented manner [7].

1.3.1 Architectural and Technological Challenges

The SRP system is characterized by a fragmented architectural structure and complex interdependencies between its main components, which leads to several critical issues.

Monolithic Frontend Component

The user interface, responsible for navigating the Service Request catalog and completing the associated forms, is an excessively large and complex Vue.js frontend component (nearly 11,000 lines). This architecture makes modifications, extensions, and client-specific customizations exceptionally challenging [15]. The logic for managing dynamic fields, their interdependencies, and the rules for compilation and validation are all handled at the frontend level through a long series of hard-coded conditional statements, such as the following:

```
<div v-if="field.desc == 'Domain'">
    ...
</div>
```

This, in addition to mixing business logic with presentation, represents a significant challenge for catalog configuration, as there is no centralized way to manage the fields and their relationships. Every change, therefore, requires a direct intervention on the code, making the system inflexible and difficult to maintain, especially in a context where the operator modifying the catalog is not a programmer and does not know how the frontend code is structured. For instance, if an operator wanted to add a new "Domain" field to a Service Request, this field would need to have the exact description "Domain" and type "domain"; otherwise, the Vue.js component would not render it correctly.

These dynamics make the system fragile and susceptible to frequent errors that waste time and resources for both clients and operators [15].

Overly Generic Backend REST Service

The only information about the fields returned by the backend REST service (`SRPWebSeviceREST`), besides the field name and its mandatoriness, is its *type*. It is therefore easy to imagine how this information, which should only be an indication of the component that needs to be rendered (e.g., textbox, select, radio button, etc.), can become an identifier for something much broader.

For example, a field of type `UPN` is a composite field derived from the value of another field, '`Display Name`', concatenated with an '`@`' symbol, and suffixed with the value from a select field whose options are the client's domains, provided by an external REST service (e.g., "`n.guarini@elmec.it`", where "`n.guarini`" is the value of the "`Display Name`" field, and "`elmec.it`" is the

value of the "Domain" select field).

Despite this complexity, the REST service returns only a JSON document of this type:

```
{
    "idField": 1603,
    "descField": "UPN",
    "type": "upn",
    "mandatory": true
}
```

thus delegating to the frontend the responsibility of interpreting the type and rendering the field correctly.

Dependency on the SRP Agent

The import of data from clients' Active Directories and the actual execution of Service Requests are handled by an agent (`SRPWindowsService`) installed directly on the clients' Domain Controllers. This approach presents several challenges:

- **Security and Permissions**: The agent often requires credentials with elevated privileges (e.g., `domain-admin`), which clients are increasingly reluctant to grant.
- **Reliability and Control**: The configurations on the agent and the Domain Controller are complex and difficult to manage centrally, which can cause execution errors and data synchronization issues.
- **Scalability**: The need to install, configure, and maintain an agent for each client domain represents a significant operational effort. Alternatives such as more modern provisioning systems will be evaluated.
- **Obsolete Communication Patterns**: The unidirectional communication through the *DMI Console* creates a bottleneck and complicates the management of asynchronous operations. Communication between components in the client's network and the company network is based on polling (e.g., the SRP Agent queries the server every 5 minutes), an approach that introduces latency and inefficiencies.

1.3.2 Code and Data Management Issues

Code and data persistence management is another critical aspect of the system, characterized by a lack of versioning and modern debugging tools.

Unversioned and Undebuggable Stored Procedures

The intensive use of Stored Procedures[2] in the MSSQL database represents one of the major weaknesses. These procedures contain fundamental business logic and are:

- **Unversioned**: There is no centralized version control system for Stored Procedures. Changes are made directly on the database, making it impossible to track changes, revert to previous versions, or test modifications in separate environments.
- **Difficult to Debug**: Debugging Stored Procedures is a complex and cumbersome process, often requiring direct access and specific SQL server tools, with the risk of affecting the production environment.

Inaccessible and Unversioned SSIS Jobs

SSIS packages[3] are essential for data import and export operations (e.g., AD master data). However, these jobs reside physically on the `ELMEC-CMI-MSSQL` server, and the only way to access and modify them is via a remote desktop. This approach prevents versioning and collaboration, limits accessibility by making maintenance and debugging an onerous and risky activity, and creates a *single-point-of-failure* and a strong coupling between the import/export business logic and the infrastructure.

1.3.3 Database Schema and Data Integrity Issues

The analysis of the database schema, represented in figures 1.7, has revealed a number of significant issues that affect the quality and integrity of the data managed by the SRP system.

Lack of Data Integrity and Consistency

One of the most significant problems concerns the lack of explicit referential integrity constraints. In the schema in figure 1.7, although the relationships between tables have been inferred manually, they are not defined at the database level through the use of foreign keys. This absence prevents the

[2]Stored procedures are SQL code routines, or sets of instructions, that are pre-compiled and stored within a relational database. They act as logical units of execution, encapsulating business logic directly at the database level [29].

[3]SQL Server Integration Services (SSIS) is a platform for building data integration and workflow solutions. It is a key component of Microsoft SQL Server, primarily used for Extract, Transform, Load (ETL) operations, enabling the extraction of data from various sources, its transformation, and its loading into different destinations [23].

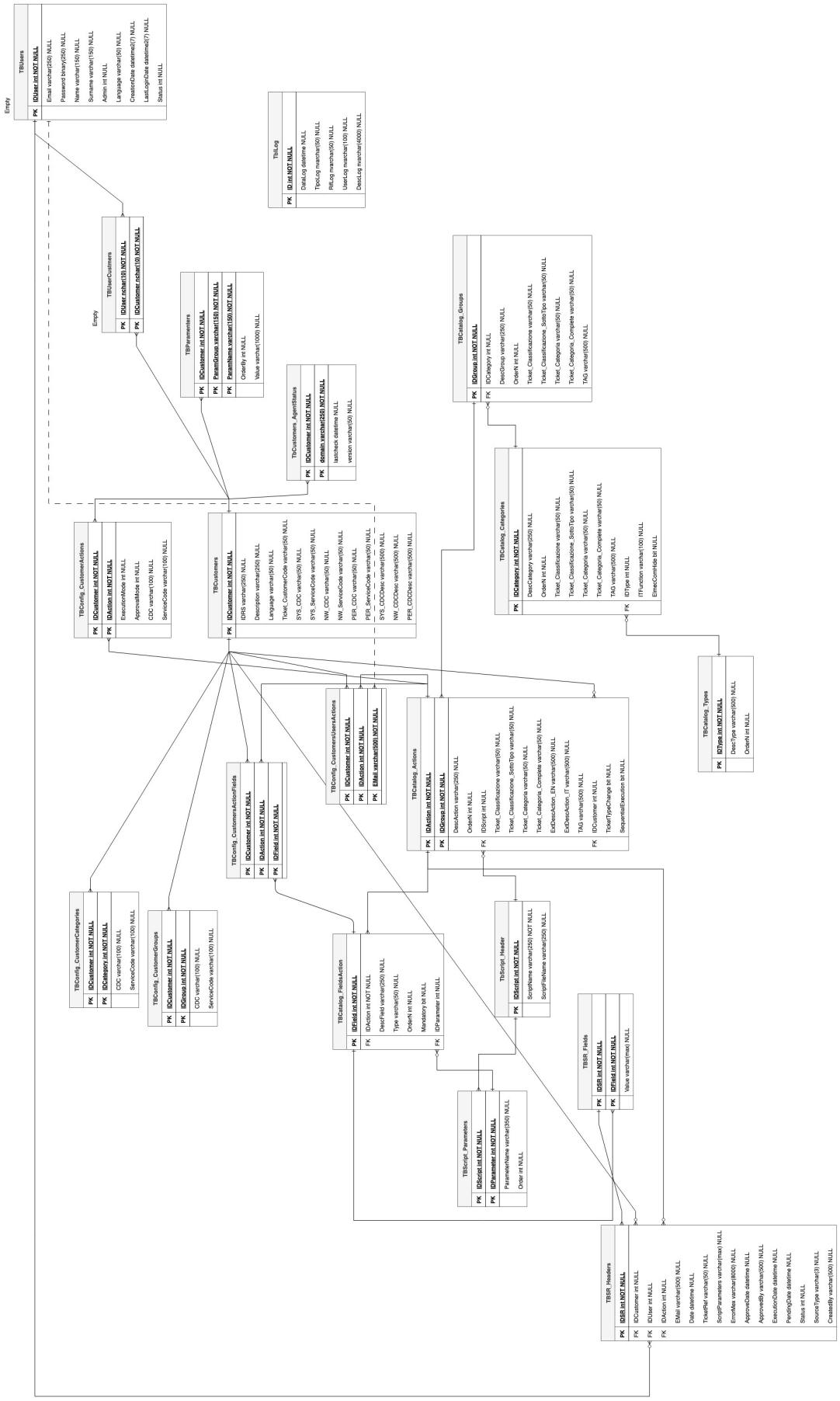

Figure 1.7: AS-IS Database ER Schema (TODO split into multiple smaller images)

database from guaranteeing data integrity [6]: for example, a row in a child table could refer to a non-existent row in the parent table [36]. Such a scenario, known as a "dangling reference[4]", can generate anomalies and inconsistencies in the system, making the data unreliable and debugging extremely complex [6].

Data Redundancy

Another issue is the high data redundancy and the large number of duplicate attributes across different tables [27]. This design, which deviates from the principles of normalization [2], leads to storing the same information in multiple locations, which not only increases the data volume but also introduces a high risk of inconsistencies [2].

Unclear and Inconsistent Naming Conventions

The naming convention for attributes and tables is inconsistent and unclear. The schema presents a mixture of conventions (e.g., `IDGroup` vs. `GroupID`, English names mixed with specific terms) and typos (e.g., `TBParamenters`), making it difficult to immediately understand the function of each field. In many cases, the naming is not explicit enough to describe the role of an attribute, requiring an in-depth knowledge of the system to correctly interpret the relationships and business logic. This lack of a unique and standardized vocabulary hinders collaboration among developers, complicates code maintenance, and increases the likelihood of errors.

1.3.4 Maintenance and Operational Challenges

According to information discussed in an internal meeting [20], the daily management of the system is made complex by several inefficiencies and obsolete design choices.

The current system design does not allow for the effective reuse of Service Requests and catalog configurations. Consequently, every modification or addition requires the creation of a new Service Request, even for small variations, leading to a innefficient and difficult-to-manage catalog, consisting of numerous duplicate SRs and fields. This, while functional, is not a scalable approach and leads to significant inefficiencies in catalog management itself. For example, if a Service Request needs to be modified for all clients who use

[4]The "dangling pointer" is a problem typically addressed in the context of low-level programming, where a pointer refers to a memory area that is no longer valid because it has been freed or because the pointer is used outside the context of the variable's existence to which it refers [11], but the same dynamic can also be applied in the context of databases, where the "pointers" are represented by foreign keys.

it, the operator will have to modify not only the SR in question but also all the duplicate SRs for each client, with the risk of forgetting a modification or making mistakes. This approach, although probably conceived to prevent error propagation, makes the catalog difficult to navigate and maintain.

The new architecture will need to address these challenges by introducing a more robust, versioned system based on modern software design principles and patterns.

Chapter 2

Architectural Redesign

Given the numerous critical issues that emerged from the analysis of the current system, it is clear that a thorough redesign is necessary. This redesign must be able to perform the system's current functions more efficiently and in an optimized manner, while also satisfying a series of requirements that did not exist when the system was developed several years ago, and which the current system cannot manage because it was not designed to do so.

2.1 New System Requirements

The system can be divided into three main components: **Catalog Structure and Management**; **Service Request Execution**, and **Master Data Import**. The requirements for each of these areas will therefore be analyzed.

2.1.1 Catalog Structure and Management

This is the most substantial component of the three, dealing with everything concerning the definition, maintenance, and configuration of the actions executable by each customer. This therefore also includes the management of form fields, customer-specific customizations, scripts, their parameters, the target machine address retrieval logic, and so on.

Before proceeding with the analysis of the requirements, a clarification on the terminology to be used is necessary: an '*Action*' is understood as the formal definition of an operation that a client can execute on its infrastructure; a '*Service Request*' (SR), on the other hand, is understood as the effective execution of an Action. If a parallel with OOP is to be drawn, the Action can be seen as a class, and the Service Request as an object, an instance of the aforementioned class.

Actions

The Action entity can be seen as the core of the catalog component, and it is composed of the following main elements:

- General metadata, such as the name, an extended description, tags, a type, a group, a category, and other information related to administrative

and billing aspects;

- The fields of the corresponding form that must be filled by the client for its execution, including the relative logic tied to types, data validation, dependencies between various fields (e.g., fields that, once completed, "enable" other fields), autofill (e.g., fields whose values are autofilled with values from other fields), sensitive values, and so on;
- The script that will be executed in the customer's infrastructure to perform the effective required operation;
- Script parameters, which may be field values, but also additional parameters necessary for execution (e.g., AWS credentials, 365 credentials, etc. . .);
- The target machine on which the script must be executed.

Some of these dynamics are already implemented in the current system (with all the critical issues of the case, as already discussed in the previous chapter), while others are not. The crucial point, however, is that all these configuration aspects must be customizable for specific customers and for specific actions [20].

Given that the users of this system are very large clients (Fendi, Valentino, Lavazza, to name a few), it is easy to imagine that each of them has highly specific needs, which make direct reuse of Actions across multiple clients impossible. At the same time, however, the creation of infinite duplicated Actions that share almost everything, with only small differences (as happens in the current system [20]), must be avoided.

It is therefore necessary to implement a system that provides for the possibility of defining a sort of hierarchical structure, where a "parent" action is configured, which can be "inherited" by several child actions that will specify only the modifications, all without affecting the default action.

Fields

Together with actions, fields are central components on which a large part of the system is based. As already mentioned, fields are entities that can be extremely diverse, such as text fields, selects, multiple-choice selects, radio buttons, etc. In the current system, there are 86 different field types. This number is so high because, due to how the current system is built, the "type" information is the only way the backend communicates to the frontend what type of field is to be rendered [26]. In fact, there are many different types that share the same logic, perhaps simply using two different services to retrieve the data. The effective field typologies are therefore much fewer, and it is crucial that they are identified and generalized as much as possible.

The general idea is that the logic for fields must be moved to the backend, which must return all the necessary information for the frontend to render the form: from field types, to validation regexes, to error messages, to any external services to be contacted to retrieve the options list, or to validate the data, and so on [20].

All this must obviously be compatible with the customization requirements specified in the previous section.

Regarding customizations, a typical example scenario could be the following: a customer uses an action, but for some reason requires two additional fields, which will then be needed by the script that will in turn require two additional parameters. Furthermore, for some fields, the client in question uses different validation rules and specific autofill templates. These customizations must be possible without modifying the default action and fields.

Regarding "particular" field types, the system must provide for [20]:

- Select fields whose options are customer-specific (e.g., "Domain" field, which is a select with all the customer's domains);
- Select fields whose options are global for the entire system (e.g., "Country" field, which provides a list of states that are the same for all customers);
- Select fields whose options are customer-specific and to obtain them an external service must be contacted, with the possible option to search (e.g., "Server" field, which is a select with all the customer's servers, returned by the Atlantis CMDB service);
- Autofilled text fields, meaning their values are obtained through a templating system (Jinja) that uses the values of other fields (e.g., "Logon Name" field, populated by concatenating the user's first and last name separated by, for example, a dot);
- Text fields whose validation occurs through regex (e.g., "Date" field, which must comply with the standard format);
- Text fields whose validation occurs through an external service (e.g., "Logon Name" field, which requires contacting the relevant Active Directory service for validation that a user with the same logon name does not already exist);
- Fields with sensitive values, whose values will therefore need to be saved encrypted, using some symmetric encryption algorithm (e.g., "Password" field);
- "Mixed" fields, composed of multiple "sub-fields" with different characteristics (e.g., "UPN" field, which is composed of the first letter of the

first name, the last name, an at sign (`@`), and the value of a select whose options are obtained from an external API call);

It is important that the system is designed in a scalable way, so that if another field typology were to be added in the future, it can be implemented with the minimum possible effort.

Configuration and Execution Consistency

A system for monitoring the consistency of the catalog configurations [20] will be necessary, in order to prevent the creation of invalid configurations, which would then lead to errors during SR execution.

For example, if a `select` type field is specified for an action, whose options are obtained from customer-specific parameters, but those parameters have not been defined for that customer, the system must report this inconsistency through some alerting system (e.g., email, ticket, monitoring dashboard). Alternatively, if an action is configured to be executed on a certain type of machine but the field necessary to select it is not among the action's fields, the system must report this inconsistency.

Similarly, a system for monitoring the consistency of SR executions must be implemented, in order to be able to report any errors that may have occurred during execution, such as an SR stuck in an "intermediate" state (e.g., `NEW`) for too long, or an SR that started execution (i.e., in `PENDING`) but never reached completion (i.e., in `COMPLETED` or `ERROR`) within a certain time limit (e.g., 2 hours).

Additional script parameters

Another topic that must be managed is that of script parameters. In the old system, there was granular control over every single parameter, which was linked to an action's script and possibly to a field in the corresponding Service Request. In practice, all the action's fields with their respective values were passed as parameters to each script, plus some additional parameters not linked to any field (e.g., Exchange Server, AD Sync Server, various credentials). This dynamic of additional parameters occurred through the definition of parameters with the foreign key on the field set to `NULL`, which were then intercepted by the stored procedure relating to the generation of the parameter string which, with hard-coded `if` statements (e.g., `IF @ParamName = 'ServerExc' BEGIN ...`), were valued accordingly, retrieving the information with custom and non-standardized logic.

In the new system, it will be necessary to standardize these dynamics, finding a way to define the additional parameters that will be needed, beyond those of the fields, during action configuration, and to generalize the implementation logic as much as possible, so that if new "groups" of additional parameters

(e.g., AWS credentials, O365 credentials, etc.) were to be added, it can be done in the most efficient way possible.

Target Machine Configuration

In the old system, Service Requests were executed exclusively on the customers' Domain Controllers. Specifically, each DC ran an agent, which periodically checked via *polling*[1] if there were any Service Requests in the `PENDING` state with a matching `domain` field.

This part of the architecture will also need to be redesigned, as it is inefficient and highly limiting by design [33]. Therefore, in addition to redesigning these dynamics, it will also be necessary to provide the option to choose, during the configuration of an Action, to execute the related SRs on other types of machines, such as internal Elmec workers, or other types of machines on the customer's network other than the Domain Controller (e.g., Exchange Server). It must also be possible to choose to execute SRs through the old flow, in order to continue supporting both systems, at least initially [20].

2.1.2 Service Request Execution

In the current system, the execution of SRs occurs according to this sequence of main events:

1. The frontend sends the request to create a new SR to the REST service (`SRPWebServiceREST`), with all the necessary data (completed fields, customer, action, etc.) [26];

2. The SR and the relevant field values are created in the database, setting the status to `NEW` [26];

3. An SSIS job, every minute, checks if there are SRs in the `NEW` state, and if any are found, they are processed, moving them to subsequent states (e.g., `WAITING FOR APPROVAL`, `PENDING`, `ASSIGNED`, etc.) depending on the action and customer configuration [21];

4. When the SR is in the `PENDING` state, it means it is ready to be executed by the agent on the customer's DC;

5. Every 10 seconds, the agent on the customer's DC checks if there are SRs in the `PENDING` state for its domain (through calls to the SOAP service (`SRPWebService`)) [27];

[1] *Polling* is a communication technique where a system or device periodically and actively samples the status of an external resource or device to check for readiness or new data.

6. If any are found, for each one, the corresponding script is downloaded, executed, and the SR status is updated to `COMPLETED` or `ERROR` depending on the execution outcome (through the SOAP service) [27];

It has already been discussed in the previous chapter how this flow has numerous criticalities, which make it inefficient and not very scalable. It will therefore be necessary to completely redesign it, taking into consideration the possibility of executing SRs on machines other than DCs, and the possibility of executing them also on machines internal to Elmec, and even virtual machines [20].

Elmec already has an internal *provisioning* system, called *Provision Prime*, which is responsible for executing Ansible playbooks on machines (internal, external, and also virtual) [16], and which could be leveraged for this purpose. It will therefore be necessary to analyze this system, and understand if and how it can be integrated into the new SR execution flow, in order to avoid the polling of agents on the DCs, and to have a generally more reliable and efficient system [20].

A constraint that must be respected, however, is that the action scripts are all in PowerShell, and it is not possible to modify them. This is because they are very numerous, have been developed over the years, have been tested and validated for every single client, and are maintained by different Elmec departments. Therefore, even if the SR execution flow is redesigned, the scripts must be executed as they are. This obviously does not apply to new scripts that will be developed in the future, which can be designed leveraging the new technologies [20].

2.1.3 Master Data Import

Another important component of the system is the one responsible for importing customer master data, necessary for populating some action fields (e.g., UPN domains, user accounts, groups, Organizational Units (OU)[2], databases, etc.). The main difficulty lies in the fact that this information physically resides on the customers' machines (large enterprise clients, with thousands of user accounts and groups, are being discussed), and therefore their retrieval in real-time is not immediate.

In the current system, this import occurs in a manner very similar to the SR scenario, but in reverse. It happens through an agent running on the customers' DCs, which periodically (hourly) (downloads and) executes a series

[2]In Windows Server, an Organizational Unit (OU) is a container within Active Directory (AD) used to logically group objects like user accounts, computer accounts, and security groups, similar to a folder in a file system [1].

of PowerShell scripts to retrieve the necessary information, and deposits them in a specific system folder (`D:\SRPortal\Anag\`) [27]. These files are then consumed by an SSIS job (one for each type of master data) that imports them into the database. The REST services then expose APIs to retrieve this information, which are used by the frontend to populate the action fields [26].

In the new system, it will be necessary to redesign this flow as well, in order to eliminate the dependence on agents on customer DCs. Even in this case, however, the PowerShell scripts cannot be modified [20], and must be executed as they are now. It will therefore be necessary to analyze how to integrate the execution of these scripts into the new flow, perhaps also leveraging Elmec's internal *provisioning* system (*Provision Prime* [16]) in this case, in order to obtain a more traceable, reliable, and efficient system [20].

2.2 New System Architecture

After gathering and defining the requirements for the new *ServiceRequestManager (SRM)* system through various meetings with stakeholders and analysis of the current system (SRP), it has been possible to design a new architecture for the system that allows for efficient and scalable satisfaction of these requirements.

The proposed new architecture for the SRM system, shown in Figure 2.1, is based on a **microservices architecture**, which allows for greater modularity, scalability, and ease of maintenance compared to the distributed-monolithic architecture of the current SRP system [8].

2.2.1 Main Components

The SRM system architecture is composed of the following main components.

Frontend

Two separate web frontends will be developed for end-users and system administrators. These frontends will communicate with the REST services via HTTP/HTTPS APIs.

Due to the business logic being moved to the REST services, the frontends will be lighter and easier to maintain, allowing for greater flexibility in implementing new functionalities and user interface improvements [15]. This also allows for the future integration of a mobile application, which can use the same APIs.

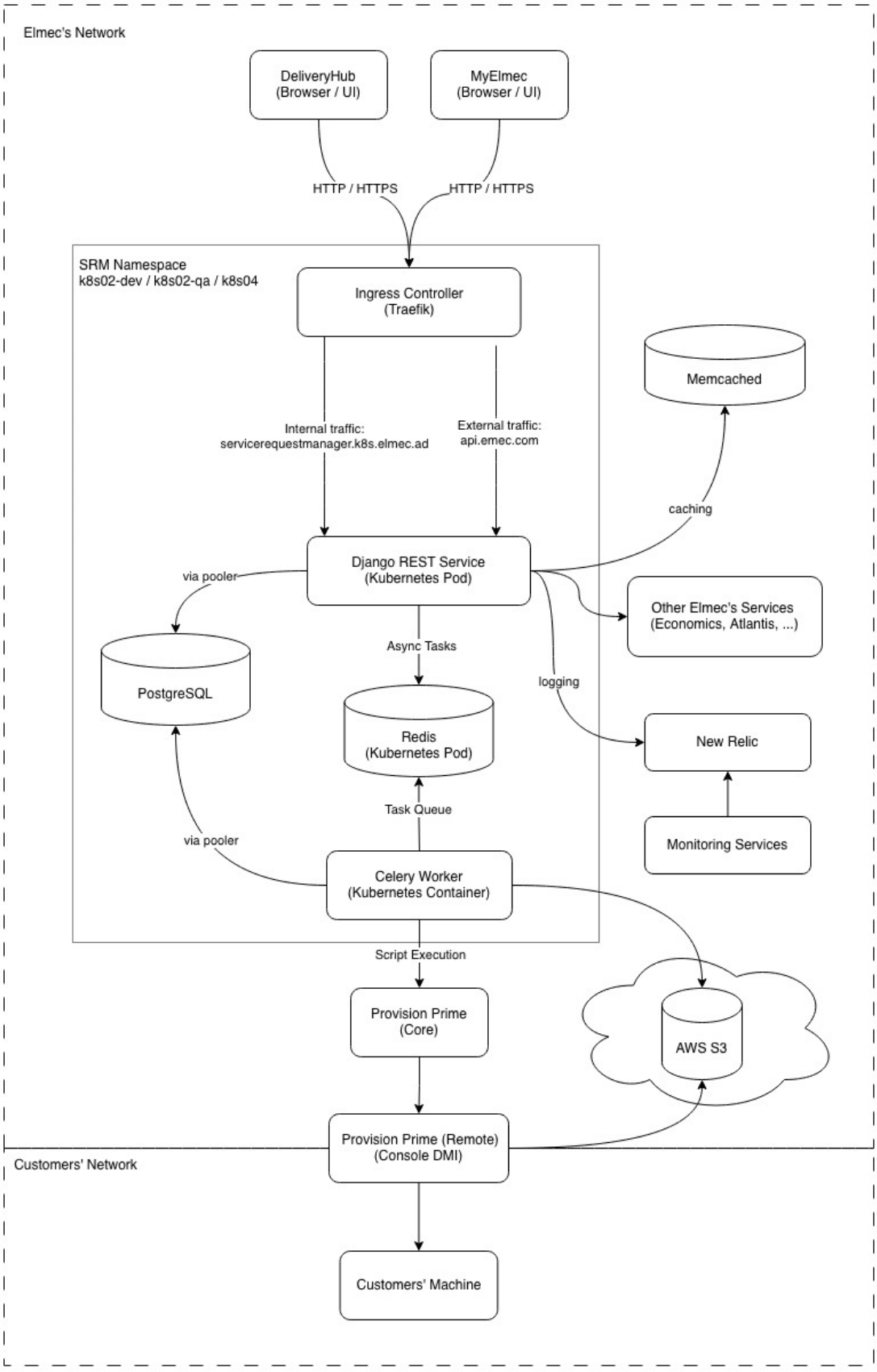

Figure 2.1: SRM System Architecture

REST Services

REST (Representational State Transfer) is an architectural style for designing web services that leverages HTTP protocols for communication between client and server [10]. The basic idea is that a system's resources (users, orders, documents, etc.) are exposed through a uniform interface, typically HTTP, using [10]:

- URLs to identify resources (e.g., `/api/catalog/actions/123/`);
- HTTP verbs to operate on resources (`GET`, `POST`, `PUT`, `DELETE`, ...);
- Representations of resources in standard formats (JSON, XML, ...).

These services will be responsible for business logic, user request management, and interaction with databases and other system components.

Authentication and Authorization System

Authentication and authorization will be managed through the internally developed library, PyElmecAuth or GoElmecAuth (depending on the implementation language), which will interface with the corporate identity management system to ensure secure and controlled access to the REST services.

It will therefore be necessary to integrate a granular permission system into SRM to ensure that users can only access the resources and functionalities for which they are authorized, based on the roles defined by the corporate Identity Provider.

Caching System

A caching system will be required to improve system performance and reduce the load on the databases.

In particular, responses from calls to external services will be cached, such as third-party APIs needed to validate some fields, or calls to retrieve customer information, to reduce response times and improve the user experience. The time-to-live (TTL) of the caches must be configurable based on the specific needs of each data type.

Message Broker and Task Queue

To manage asynchronous operations and improve system scalability, a Message Broker (e.g., RabbitMQ, Apache Kafka, Redis, ...) will be used in combination with a Task Queue (e.g., Celery).

This is necessary as the SR lifecycle is very complex and involves many operations that can require long execution times [19], such as sending emails,

maintaining HDA[3] tickets, integration with external systems, etc. By using a messaging and task queue system, these operations can be executed in the background, without blocking the application's main flow [30].

Provisioning System

It will also be necessary to integrate a provisioning system to execute the scripts related to SRs and those for master data import directly within the clients' networks. This system must be secure, reliable, and scalable, in order to be able to manage a large number of provisioning requests simultaneously.

Elmec already has its own provisioning system called *Provision Prime* [16], which will be integrated with the new SRM system to execute scripts efficiently and securely.

Monitoring and Logging System

To ensure the stability and reliability of the system, a monitoring and logging system will be required. This system will allow tracking system performance, identifying any problems, and analyzing logs to resolve bugs. Tools such as Prometheus, Grafana, New Relic, or other similar tools will be used to implement this system.

Both system metrics (CPU, memory, database usage, API response times, etc.) and application logs (errors, warnings, debug information, etc.) will be tracked, in order to always have a complete view of the system status, and to have the possibility of launching automatic alarms in case of problems (e.g., if the number of responses resulting in 500 (Internal Server Error) in the last hour is $\geq 1\%$).

Containerization and Container Orchestration System

To facilitate system deployment, scalability, and management, a containerization system (Docker) will be used in combination with a container orchestration system (Kubernetes).

This will allow isolating the different system components, simplifying the deployment process, and ensuring greater resilience and scalability of the system, through functionalities such as replicas, chronjobs, automatic scaling of pods, and so on [14].

[3]HDA (Help Desk Advanced) is a web-based ticketing and service management system developed by Zucchetti. It enables organizations to manage tickets, incidents, and support processes through a centralized platform [12].

2.3 Addressing the Identified Challenges

All the challenges described in section 1.3 have been analyzed, addressed, and resolved through a series of targeted interventions. These interventions concerned various aspects of the system, from restructuring the software architecture and revising data management processes to implementing new technologies and development practices. The solutions adopted for each of the identified challenges are detailed below.

2.3.1 Architectural and Technological Challenges

The adoption of an approach based on containerization (with Docker) and orchestration (with Kubernetes, often abbreviated as K8s) forms the foundation for overcoming the current architectural and technological limitations of the SRP system. This transition not only directly addresses issues of modularity, isolation, and dependency management but also paves the way for a more modern and scalable architecture, such as microservices or micro-frontends. Specifically, it aims to solve the problems of fragmentation, dependency complexity, and maintenance difficulties inherited from the previous monolithic architecture and the agent installed at the customer's site.

Containerization with Docker

Docker is a containerization platform that allows developers to package applications and all their dependencies (libraries, configurations, environments) into a single, standardized unit called a container. As illustrated in figure 2.2, these containers are isolated from each other but share the host operating system's kernel, making them extremely lightweight and portable.

The use of Docker resolves the "works on my machine but not in production" problem (known as "dependency hell") [3] and provides a solid foundation for component decoupling. Each component (frontend, backend, support services) will be enclosed in a *Docker container*, and ensures that the execution environment (including libraries, configurations, and dependencies) is identical across development, testing, and production [3]. This eliminates errors caused by environmental discrepancies.

Once created, the container can be run anywhere Docker is present (in our case, container orchestration is managed by Kubernetes, which will be discussed in the next subsection), simplifying the deployment process. The system thus becomes extremely more portable, moving away from the strict dependence on specific operating systems and hardware configurations like, in this case, Windows Server and Microsoft SQL Server.

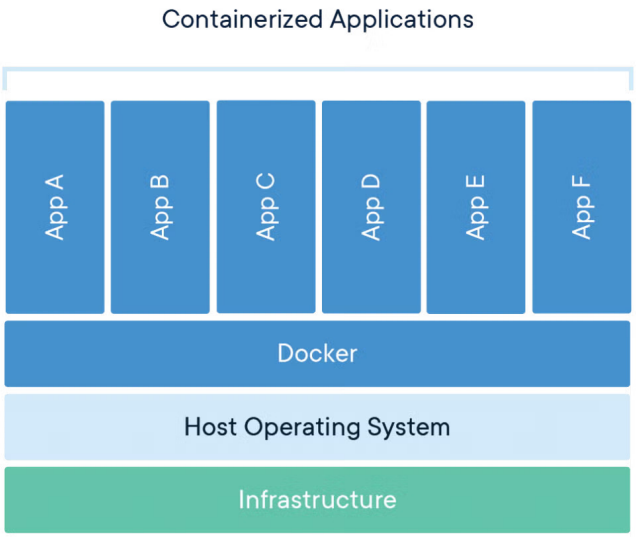

Figure 2.2: Docker Container Architecture (source: [35])

Orchestration with Kubernetes

Kubernetes (K8s) is an open-source system for automated container orchestration, originally developed by Google. Its purpose is to manage entire fleets of Docker containers in a *cluster*, ensuring that applications are always available, scalable, and resilient [14].

If Docker provides the mechanism for creating and distributing containers, Kubernetes is its natural complement, handling their large-scale management (*orchestration*), transforming the infrastructure from a set of single machines into a centrally managed cluster [14].

K8s allows components to scale horizontally based on the workload (via the Horizontal Pod Autoscaler). When demand increases, K8s automatically launches new instances of services (e.g., more pods for the REST backend) and removes them when the load decreases. This is crucial for managing traffic peaks [18].

In parallel, it constantly monitors the status of the containers. If a component fails, K8s automatically restarts it or schedules a new one on a healthy node in the cluster, ensuring high availability and resolving the single-point-of-failure problem introduced by the current dependence on single servers and the agent [18].

Kubernetes also supports advanced deployment strategies (e.g., Rolling Updates), allowing code updates without service interruptions (zero-downtime deployment) [18]. In case of issues, rolling back to the previous version is fast

and automated. This mitigates the risk associated with system changes.

Finally, K8s also offers native mechanisms to manage environment variables and sensitive data (such as database credentials or external service authentication tokens) separately from the container code [18]. This centralizes management and increases security, eliminating the numerous hard-coded configurations present in the old system and making the containers more generic.

The adoption of Docker and Kubernetes therefore creates the ideal environment for the subsequent decomposition of the frontend monolith and the re-engineering of the various backend components, topics that will be covered in detail in the following sections.

Monolithinc Frontend Component

At a technological level, the chosen technology did not change compared to the existing system, which is Vue.js. However, Vue.js version 3 was used, which introduces numerous improvements compared to version 2 used in the existing system, including better performance management, a more flexible composition system, and improved support for TypeScript [34]. In fact, in addition to the updated version of Vue.js, it was decided to adopt TypeScript as the development language for the frontend, in order to improve code maintainability and quality [32].

TypeScript Developed in 2012 by Microsoft as free and open-source software (Apache License 2.0) [32], TypeScript extends JavaScript's functionalities by introducing optional static typing. The main advantage is that, unlike JavaScript, where type errors are detected only at runtime, TypeScript allows most errors to be caught already at the *build time* stage. This ensures greater code robustness and a significant reduction in production bugs. For extended projects like this one (the Service Requests component is actually only a part of a much larger customer-dedicated platform called *MyElmec*), static typing significantly improves code maintainability and readability, acting as a form of implicit documentation and allowing for safer and faster refactoring.

To solve the maintainability and scalability problems of the existing monolithic frontend, it was decided to move all the configuration and customization logic for field behavior (autofill, dynamic fields via external APIs, validation, dependencies, etc.) to the backend (a topic that will be explored in section 2.3.1). In this way, the frontend was transformed into a simple interface for display and user interaction (as it should be), significantly reducing code complexity and facilitating the implementation of new functionalities. In particular, the frontend is now limited to reading a JSON configuration provided by the backend, which describes the form structure, the fields to be displayed, their properties, and validation rules. This was also made possible thanks to the

potential offered by Vue.js 3, which allows for the creation of highly dynamic and reusable components based on external data [34].

This led to a transition from a total of 86 different "types" of fields in the existing system to only 10 generic field types:

- Text
- Numeric
- File
- Password
- Checkbox
- Select (with already defined options)
- Select (with API-fetched options)
- Date / DateTime
- UPN (managed specifically due to its peculiarities, as it is composed of two sub-fields and has a particular validation logic)
- ADOU Tree

All the various customization logics (especially the customer-specific ones) thus become transparent to the frontend, which merely interprets the configuration provided by the backend. The system transitioned from a component with approximately 11,000 lines of code to a component with only 1,000 lines of code, resulting in improved maintainability and ease of system extension, and with the possibility of implementing other frontends (e.g., a mobile app) with minimal effort without having to replicate all the complex field management logic.

Overly Generic Backend REST Services

To solve the issues related to the frontend, a deep restructuring of the backend logic was necessary, especially concerning fields, moving from JSON responses of this type

```
{
    "idAction": 455,
    "idField": 1649,
    "descField": "Manager",
    "type": "ADUser",
    "idCustomer": 42,
    "orderN": 53,
```

```
    "mandatory": false
}
```

to much more detailed and specific responses, which provide the frontend with all the necessary information for the complete rendering of the entire form, such as the following:

```
{
    "field": {
        "id": 3001,
        "name": "manager",
        "description": "Manager",
        "type": "select",
        "is_sensitive": false,
        "option_source_policy": "EXTERNAL_API",
        "external_api_config": {
            "name": "AD_USERS",
            "base_url": "https://q-api.elmec.com/srp/",
            "endpoint_pattern":
             ↪  "api/customer/VFG/adusers?domain={{domain}}",
            "http_method": "GET",
            "headers_json": {
                "Authorization": "{{TOKEN_TYPE}} {{TOKEN}}"
            },
            "response_mapping_json": {
                "name": "userDesc",
                "value": "user"
            },
            "timeout_seconds": 10,
            "is_searchable": true,
            "search_param_name": "search",
            "root_parent_id": null
        },
        "options": null,
        "multiple_choice": false,
        "file_accept_mimes": null
    },
    "mandatory": false,
    "validation_regex": "^[a-zA-Z0-9._]{5,20}$",
    "validation_regex_message": "Invalid format for Manager
     ↪  field.",
    "autofill_template": null,
    "depends_on": [
        "domain",
        "first_name",
        "last_name"
```

```
38      ],
39      "display_order": 26
40  }
```

In addition to modeling all possible properties of a field in a clear and structured way, the new REST services are also responsible for managing all the customization and configuration logic of the fields, with a three-level structure (which will be detailed in the chapter, dedicated to implementation aspects):

1. Fields
2. Actions
3. Customers

Broadly speaking, the idea is that each field (first level "Fields") has a set of generic properties (type, description, options, etc.) that can be further customized based on the specific action (second level "Actions") to which the field belongs (e.g., making it mandatory or not, adding validation rules, autofill logic, dependencies on other fields, etc.). Finally, each action can be further customized per customer (third level "Customers"), allowing the field's behavior to be adapted to the specific needs of each customer (e.g., modifying the available options in a selection field, changing validation rules, etc.), all without affecting the "original" configuration of the underlying level.

This three-level structure allows for a high degree of reusability and modularity, making it possible to define generic fields that can be easily adapted to different situations without having to duplicate code or logic.

Dependency on the SRP Agent

The dependency on the agent installed at the customer site has been completely eliminated thanks to the integration of Provision Prime, an internally developed provisioning tool written in Go, which manages the execution of Ansible playbooks on remote machines, in this case, the servers within customer networks, transparently to the developer (who will simply configure the playbook and make an API call).

Ansible Ansible is an open-source, agentless IT automation engine used for configuration management, application deployment, intra-service orchestration, and provisioning [4]. It is designed to be simple, powerful, and easy to use, operating over standard SSH for Linux/Unix and WinRM for Windows, thus requiring no special client software (agents) to be installed on the managed nodes [4]. This reliance on existing transport protocols and its human-readable structure, written in YAML, contribute to its minimal learning curve and rapid adoption. The core of Ansible's functionality are

Playbooks. A Playbook is essentially a blueprint of automation tasks, structured as a YAML file, that describes a set of steps to be executed on specified hosts or groups of hosts (defined in an inventory) [4]. They define the desired state of a system in a clear, declarative manner. Each Playbook consists of one or more plays, and each play is an ordered list of tasks. A task is a call to an Ansible module, which is the unit of code that performs a specific action, such as installing a package, copying a folder, or executing scripts. Playbooks ensure that complex, multi-tier application deployments and job executions are repeatable and reusable, transforming manual, error-prone processes into reliable, version-controlled automation.

In this way, not only is the dependency on the agent, and all the associated logic, eliminated, but all the scripts related to the Service Requests, which previously resided physically inside the customers' Domain Controllers and were not tracked in any way, are now versioned.

The cost of these improvements is represented by the additional logic needed to manage communications with Provision Prime, specifically the retrieval of the target machine. To implement these retrieval logics, which are highly custom for each type of target machine (e.g., Domain Controller, Exchange Server, internal workers, etc.), the *Strategy* design pattern [17] was leveraged. This pattern allows each retrieval logic to be encapsulated in a separate class, making the code more modular, extensible, and maintainable (a topic that will be explored in depth in chapter X.X).

As for the script execution logs, which are needed to monitor the status of Service Requests and detect execution errors, an integration with an AWS S3 bucket, a scalable and durable storage service, will be implemented. This way, every time a script is executed on a remote machine, the logs will be uploaded by the DMI (via the Ansible playbook) to the S3 bucket, ensuring centralized and secure access to execution logs regardless of the machine on which they were generated.

2.3.2 Code and Data Management Issues

The lack of versioning and modern debugging capabilities outlined in the existing system is fundamentally addressed by centralizing all disparate logic (previously scattered across Stored Procedures and SSIS jobs) into a unified, version-controlled software project integrated with a robust CI/CD pipeline. By adopting Git as the standard Version Control System (VCS), all business logic is now treated as application code. This immediately solves the versioning problem: every change is tracked, attributed to an author, and can be reverted, enabling seamless collaboration and a full audit trail. Moreover, moving the core business logic out of the database and into a dedicated backend service (Django REST Framework) written in a modern language (Python) drastically improves debuggability, allowing developers to use powerful, standardized IDE

tools to set breakpoints, inspect variables, and step through code without impacting the production database.

This strategy is completed by integrating several specific CI/CD pipelines (a topic that will be explored in depth in chapter X.X) tailored for each environment branch (`feature`, `devel`, `quality`, `main`). This setup automates the deployment process: code changes are first pushed to a dedicated task branch, reviewed, and then merged into the `devel` environment. Upon passing, the code is merged to the `quality` environment, and finally to `main` for production release. This process ensures that no logic is directly modified in a production environment, eliminating the single-point-of-failure inherent in direct database and server access. Furthermore, replacing the inaccessible, unversioned, and tightly coupled SSIS Jobs with modern, containerized async tasks (managed as code within the Git repository) breaks the strong infrastructure coupling, enhancing accessibility, facilitating collaboration, and making maintenance and debugging a safe, environment-specific activity.

2.3.3 Database Schema and Data Integrity Issues

The issues related to database design and data management were addressed through a series of interventions aimed at improving data integrity, consistency, and maintainability. These interventions include a complete revision of the database schema to eliminate redundancies and inconsistencies, the implementation of stricter integrity constraints, and the adoption of clear and consistent naming conventions for tables and fields.

The system transitioned from using Microsoft SQL Server as the Database Management System (DBMS) to PostgreSQL, an open-source DBMS known for its robustness, scalability, and compliance with SQL standards. PostgreSQL offers advanced features such as support for complex data types, full ACID transactions, and an extensibility system that allows new functionalities to be added via extensions [28]. This choice not only improves the system's performance and reliability but also facilitates data management and the implementation of database design best practices.

Lack of Data Integrity and Consistency

To resolve data integrity and consistency issues, stricter referential integrity constraints were implemented using well-defined primary and foreign keys, also specifying behaviors in case of deletion (`ON DELETE CASCADE` / `SET_NULL`, etc...).

Specifically, for the deletion of "core" entities (such as customers, actions, or fields), the `CASCADE` behavior was chosen for "child" entities, which are consequently deleted.

For secondary entities and/or those that can be null (such as ticket categorization information, action categories and groups, order information associated with tickets, etc...), the `SET_NULL` behavior was chosen to preserve the history of operations and associated data, preventing the loss of more important information.

These constraints ensure that relationships between tables are correctly maintained, preventing the creation of orphaned or inconsistent data.

Data Redundancy

New relationships were created to maximize data reuse and reduce redundancy, while ensuring referential integrity through the use of well-defined primary and foreign keys.

These new entities primarily model concepts that were previously duplicated across multiple tables, such as cost centers (CDC, *"centro di costo"*), service codes, and ticket classification and category. This significantly reduced the amount of redundant data, improving storage efficiency and database maintainability.

Unclear and Inconsistent Naming Conventions

Now, all tables and their attributes follow standardized naming conventions (*snake_case*), improving the readability and maintainability of the database schema.

For "core" relationship names, the plural form was chosen (e.g., `actions`, `service_requests`, `customers`, etc...), while for "child" relationships, the singular name of the "parent" relationship followed by the specific name in the plural was chosen (e.g., `action_fields`, `customer_actions`, etc...).

2.3.4 Maintenance and Operational Challenges

With the new improvements introduced in this chapter concerning architecture, technology, the completely redesigned catalog and configuration management, and the new execution flow, many of the previously encountered maintenance and operational problems have been resolved. The standardization of development and deployment processes, the adoption of DevOps practices, and the implementation of a more modern and scalable infrastructure have led to a significant reduction in downtime, while improving the system's overall reliability. Furthermore, the adoption of advanced monitoring and logging tools allows for rapid identification and resolution of any issues, ensuring smooth and continuous system operation.

These improvements not only facilitate daily operations but also long-term

planning, enabling the system to adapt easily to future needs and challenges.

Chapter 3

Implementation of a modular and scalable solution

Se il capitolo precedente ha descritto in dettaglio i requisiti e il nuovo progetto architetturale per il nuovo sistema ServiceRequestManager (SRM), proponendo soluzioni mirate per superare le varie limitazioni tecnologiche e strutturali del sistema legacy SRP, questo capitolo segna il passaggio dal piano teorico alla realizzazione pratica, concentrandosi sull'implementazione concreta di una soluzione modulare e scalabile.

Il focus primario di questa implementazione risiede nel modo in cui sono state risolte le sfide critiche descritte nei capitoli precedenti. Nello specifico si evidenzierà come la centralizzazione della logica di gestione e configurazione dei campi (e più in generale del catalogo) sul backend abbia eliminato il frontend monolitico e reso possibile un sistema di configurazione a tre livelli altamente riutilizzabile. Si illustrerà inotlre l'integrazione del nuovo flusso di esecuzione e retrieve delle anagrafiche tramite sistema di code di messaggi, essenziale per l'eliminazione della dipendenza dall'obsoleto SRP Agent.

3.1 Automation of the Development Lifecycle through CI/CD Integration

Per ogni funzionalità o modifica significativa del codice, viene creato un branch dedicato denominato con l'ID del ticket associato alla modifica (es: IN-7835). Questi branch sono utilizzati per lo sviluppo isolato delle nuove funzionalità, permettendo ai team di lavorare in parallelo senza interferenze. Una volta completata la modifica, il branch viene sottoposto a revisione del codice e test automatici tramite la pipeline CI/CD prima di essere integrato nel branch principale (main).

Esistono 3 ambienti live distinti: `devel`, `quality` e `production`. Il Git workflow aziendale prevede che i branch di funzionalità vengano prima mergiati in `devel` per ulteriori test e integrazioni, e successivamente in `quality` per la validazione finale prima del rilascio in produzione. Questo approccio garantisce che ogni modifica sia accuratamente verificata e testata in ambienti progressivamente più vicini alla produzione, riducendo il rischio di introdurre bug o

regressioni nel sistema live.

3.1.1 Feat Branches

Per i feat branches, la pipeline CI/CD è progettata per eseguire una serie di controlli automatici volti a garantire la qualità, la sicurezza e la conformità del codice prima della sua integrazione nei branch principali, non includendo quindi le fasi di build, test e deploy dell'applicativo, come mostrato in figura 3.1.

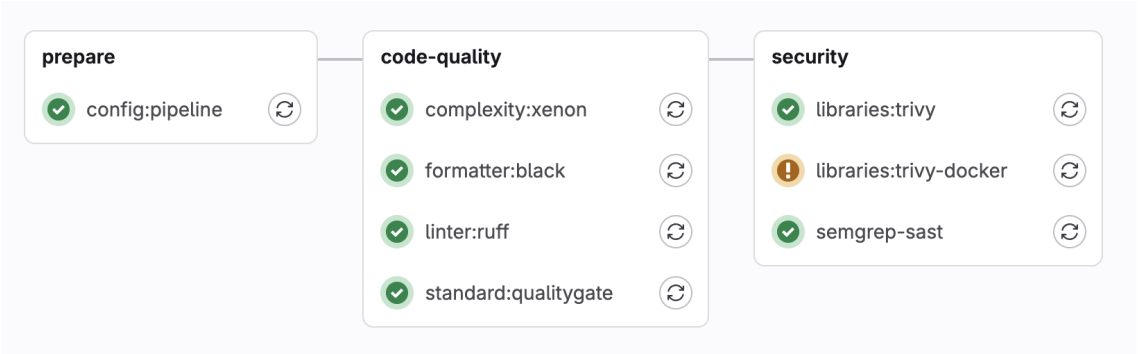

Figure 3.1: Feat Branches pipeline workflow

Prepare

Lo stage `config:pipeline` ha il compito di preparare l'ambiente di esecuzione recuperando dinamicamente la configurazione del progetto da un servizio esterno. Dopo aver installato gli strumenti necessari, esegue una richiesta HTTP per ottenere i parametri di configurazione in formato JSON e li converte in variabili d'ambiente standardizzate, salvandole in un file .env. Tale file viene poi pubblicato come *artifact*, in modo che gli step successivi della pipeline possano utilizzare queste informazioni per adattare il proprio comportamento alle impostazioni specifiche del progetto.

Code Quality

Complexity Check Lo stage `complexity:xenon` si occupa di misurare la complessità del codice tramite l'analizzatore statico *Xenon*, con l'obiettivo di identificare file o funzioni che potrebbero risultare difficili da manutenere. Dopo aver predisposto l'ambiente Python ed installato lo strumento necessario, il job esegue l'analisi confrontando i risultati con una soglia di qualità predefinita. L'esito viene salvato in un file di report e inviato al sistema di monitoraggio della qualità del progetto. Se vengono rilevate criticità che superano il livello consentito, il job segnala un fallimento, contribuendo così a mantenere la complessità del software entro limiti sostenibili nel tempo.

Code Formatter Lo stage `formatter:black` ha lo scopo di verificare che il codice Python rispetti le convenzioni di formattazione stabilite dal pro-

getto (PEP-8), utilizzando lo strumento *Black*. Durante il job viene scaricata la configurazione di formattazione condivisa, quindi viene eseguito un controllo sull'intero repository senza applicare modifiche automatiche. L'esito dell'analisi viene salvato in un report e registrato nel sistema di monitoraggio della qualità. Se vengono trovati file non conformi allo stile richiesto, il job segnala un errore così da garantire che tutte le modifiche rispettino standard uniformi di formattazione del codice.

Code Linter Lo stage `linter:ruff` è dedicato al controllo statico del codice al fine di individuare errori, costruzioni poco sicure o non conformi alle linee guida interne di sviluppo. Viene eseguito un linter Python (*Ruff*) che analizza l'intero progetto e produce un report con eventuali violazioni o segnalazioni di refactoring. I risultati vengono raccolti ed inviati al sistema di monitoraggio della qualità, contribuendo a mantenere uno stile uniforme e a prevenire potenziali problemi in fase di esecuzione. In caso di errori considerati bloccanti secondo le regole definite dal progetto, il job fallisce, impedendo l'integrazione di codice non conforme.

Quality Gate Lo stage `standard:qualitygate` analizza staticamente il progetto attraverso un insieme di regole che verificano la conformità agli standard aziendali. Il sistema ispeziona file Python, configurazioni YAML, requirements e script di deployment, controllando aspetti come la corretta configurazione del debug, la presenza di sistemi di monitoring (*Sensu* / *Prometheus*), l'utilizzo appropriato delle librerie standard aziendali, la configurazione dell'APM (Application Performance Monitoring), e la gestione sicura delle variabili d'ambiente. Ogni violazione delle regole viene riportata con dettagli sul file e la linea interessata, bloccando la pipeline se vengono rilevati errori critici. Questo approccio garantisce che solo codice conforme agli standard di sicurezza e qualità possa essere deployato in produzione, riducendo significativamente il rischio di problemi operativi e vulnerabilità.

Security

Libraries Lo stage `libraries:trivy` è focalizzato sull'analisi delle dipendenze allo scopo di individuare vulnerabilità note nei pacchetti utilizzati dal progetto. Viene utilizzato Trivy, un tool specializzato nella scansione di librerie e componenti software rispetto a database aggiornati di CVE [31]. Il risultato del controllo permette di identificare tempestivamente rischi di sicurezza legati a versioni affette da bug o falle note [31], contribuendo così a mantenere un livello di sicurezza adeguato e a prevenire l'introduzione di dipendenze vulnerabili nel processo di rilascio.

Lo stage `libraries:trivy-docker`, invece, si focalizza sulla ricerca di misconfigurazioni nei file di deployment, Dockerfile e manifesti Kubernetes. Trivy esegue una scansione completa utilizzando una priorità di detection "*compre-*

hensive" e genera un report dettagliato in formato JSON, che viene salvato come artifact per permettere analisi successive. La scansione verifica anche le licenze delle dipendenze e fallisce se vengono rilevate vulnerabilità critiche non risolte o misconfigurazioni significative.

Questi due stage forniscono un feedback essenziale sulla postura di sicurezza del progetto, permettendo al team di identificare e correggere proattivamente potenziali problemi di sicurezza prima del deployment in produzione.

Static Application Security Testing (SAST) Questo stage esegue un'analisi statica del codice sorgente per identificare vulnerabilità di sicurezza e pattern di codice potenzialmente pericolosi. *Semgrep* analizza il codice senza eseguirlo, rilevando problematiche come SQL injection, XSS, gestione insicura di credenziali e altre vulnerabilità OWASP [22]. Lo stage genera un report standardizzato in formato JSON che classifica le vulnerabilità per severity (Critical, Medium, Info) e include metadati come SHA del commit, branch e nome del progetto. Per i branch principali (quality, devel, master), i risultati vengono inviati a un sistema centralizzato di valutazione dei progetti per il tracking storico. In questo modo, si garantisce che nessun codice con problemi di sicurezza noti possa raggiungere gli ambienti di produzione.

3.1.2 Live Environment Branches

Per gli ambienti live, ovvero quelli di devel, quality e production, la pipeline CI/CD include ulteriori fasi specifiche per la build, il test e il deploy dell'applicativo, come mostrato in figura 3.2, escludendo le fasi già coperte per i feat branches.

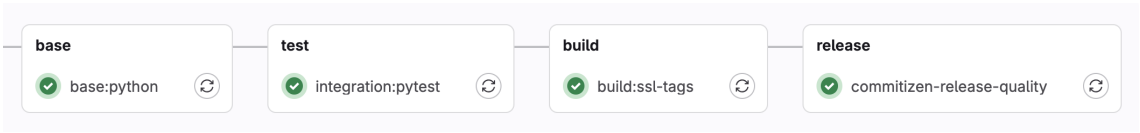

Figure 3.2: Live Environment Branches pipeline workflow (ecluding already covered stages)

Base Image Build

Lo stage `base:python` è responsabile della costruzione dell'immagine Docker di base del progetto, contenente tutte le dipendenze Python definite nel file `requirements.txt`.

Questo stage implementa una strategia di build ottimizzata attraverso la separazione in layer: l'immagine viene ricostruita solamente quando vengono rilevate modifiche nei file `requirements.txt` o `Dockerfile_base`, evitando rebuild non necessari e riducendo significativamente i tempi di esecuzione della pipeline. Il job viene eseguito esclusivamente sui branch principali (devel, quality, master).

Questa separazione in tre Dockerfile distinti (Dockerfile, Dockerfile_base, Dockerfile_ssl) consente di sfruttare la cache Docker in modo intelligente: se le dipendenze non cambiano, gli stage successivi possono riutilizzare l'immagine base esistente, velocizzando notevolmente il processo di build e riducendo il carico sui runners CI/CD.

Test

Lo stage `test:pytest` si occupa dell'esecuzione della suite di test automatizzati del progetto utilizzando PyTest e PostgreSQL come database di riferimento. Il job configura un ambiente di test isolato avviando un'istanza dedicata di PostgreSQL 16 come servizio containerizzato, con credenziali e parametri di connessione definiti dinamicamente attraverso variabili d'ambiente. L'esecuzione dei test avviene solo se è presente il file requirements_tests.txt e viene adattata in base al contesto: per i branch devel e quality vengono utilizzate rispettivamente le immagini Docker `devel-latest` e `quality-latest`, mentre per le merge request viene impiegata l'immagine `production-latest`, garantendo così che i test vengano eseguiti in condizioni quanto più simili all'ambiente target.

Lo stage implementa inoltre un controllo rigoroso sulla copertura del codice: se la percentuale di coverage risulta inferiore all'80%, la pipeline fallisce automaticamente, garantendo che venga mantenuto un livello minimo di qualità e affidabilità del software attraverso una adeguata copertura dei test. Lo stage viene escluso dai tag e dal branch master per evitare esecuzioni ridondanti.

Questo approccio assicura che ogni modifica al codice venga validata contro un database reale prima dell'integrazione, riducendo il rischio di regressioni e problemi di compatibilità.

SSL Image Build

Lo stage `build:ssl-tags` è dedicato alla costruzione dell'immagine Docker intermedia contenente i certificati SSL e le configurazioni di sicurezza necessarie per le comunicazioni crittografate del progetto. Questo stage fa parte della strategia di build multi-layer e viene eseguito esclusivamente sui branch quality e master, solo se presente il file `Dockerfile_ssl`. Analogamente allo stage `base:python`, implementa un sistema di rebuild condizionale che ricostruisce l'immagine solamente quando vengono rilevate modifiche nei file `requirements.txt` o `Dockerfile_ssl`, ottimizzando i tempi di esecuzione della pipeline attraverso il riutilizzo della cache Docker.

Questa separazione del layer SSL consente di gestire in modo indipendente le configurazioni di sicurezza e i certificati, facilitando gli aggiornamenti e la manutenzione degli aspetti legati alla crittografia senza dover ricostruire

l'intera immagine applicativa.

Release

Lo stage `commitizen-release` gestisce il processo di versioning automatico e la creazione di release del progetto utilizzando *Commitizen*, uno strumento che analizza la cronologia dei commit per determinare automaticamente il numero di versione secondo le convenzioni di Semantic Versioning[1] e Conventional Commit[2]. Lo stage `commitizen-release` opera sul branch master e, dopo aver configurato le credenziali Git e sincronizzato il repository locale con quello remoto, esegue un bump automatico della versione basandosi sui messaggi dei commit convenzionali (feat, fix, core, refactor, test, ...). Il processo crea automaticamente i tag Git corrispondenti e li pusha al repository con l'opzione `ci.skip` per evitare l'attivazione ricorsiva della pipeline. Lo stage `commitizen-release-quality` funziona in modo analogo ma è dedicato al branch quality e genera versioni pre-release con suffisso "alpha", modificando dinamicamente la configurazione di Commitizen per adattarla al contesto di test. Entrambi gli stage vengono eseguiti solo se è presente il file di configurazione `.cz.yaml` e sono esclusi dalle merge request, garantendo che il versioning avvenga esclusivamente sui branch principali dopo l'integrazione del codice.

Questo approccio automatizzato elimina la necessità di gestire manualmente i numeri di versione e assicura coerenza e tracciabilità nel processo di rilascio del software.

3.1.3 Tag Branches

Project Build

Lo stage `build:project-tags` rappresenta la fase conclusiva del processo di build, responsabile della costruzione dell'immagine Docker finale dell'applicazione pronta per il deployment. Questo stage viene attivato esclusivamente quando viene creato un tag Git che rispetta il formato Semantic

[1]Il semantic versioning è una convenzione di versionamento che adotta lo schema MAJOR.MINOR.PATCH, dove il numero MAJOR viene incrementato in caso di modifiche incompatibili con le versioni precedenti, il numero MINOR in seguito all'introduzione di funzionalità compatibili, e il numero PATCH per interventi correttivi che non alterano le funzionalità esistenti.

[2]Il Conventional Commits è una convenzione per la stesura dei messaggi di commit che utilizza una struttura standardizzata composta da un tipo di modifica (feat, fix, chore, test, refactor, ...), un oggetto opzionale e una descrizione, al fine di rendere più chiaro e automatizzabile il tracciamento delle modifiche, la generazione delle note di rilascio e il versionamento del software.

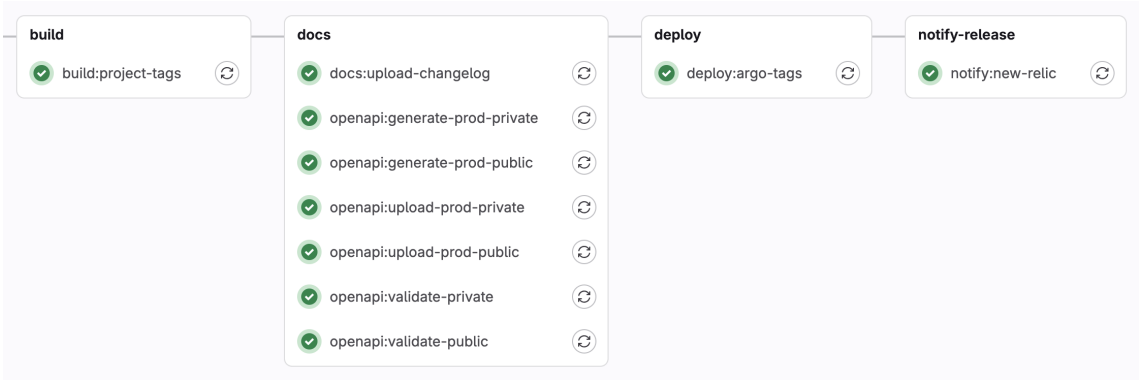

Figure 3.3: Tag Branches pipeline workflow (ecluding already covered stages)

Versioning (major.minor.patch), garantendo che solo versioni formalmente rilasciate vengano containerizzate. Il job utilizza il `Dockerfile` principale, che assembla tutti i layer precedentemente costruiti (base e SSL), aggiungendo il codice applicativo e le configurazioni finali. Lo stage implementa una logica condizionale che previene l'esecuzione se esiste un `Dockerfile_base` separato, delegando in quel caso la build a flussi più ottimizzati che sfruttano la cache multi-layer. L'immagine risultante viene taggata con la versione specifica del rilascio e pubblicata nel registry Docker aziendale, costituendo l'artifact definitivo pronto per essere deployato negli ambienti di produzione attraverso gli stage successivi della pipeline.

Docs

Gli stage di documentazione automatizzano la generazione e la pubblicazione della documentazione tecnica del progetto. Lo stage `docs:upload-changelog` si occupa di pubblicare il changelog su Techpedia, una piattaforma aziendale di documentazione collaborativa, utilizzando l'API GraphQL per creare o aggiornare automaticamente le pagine di changelog quando viene creato un nuovo tag di release. Gli stage `openapi:generate` si occupano invece della generazione della documentazione API in formato OpenAPI/Swagger, utilizzando il framework Django Spectacular per estrarre la specifica dalle annotazioni del codice. Questi stage producono due versioni della documentazione: una pubblica, contenente solo gli endpoint accessibili esternamente, e una privata con la documentazione completa di tutte le API interne. La generazione avviene per ciascun ambiente (production e quality) collegandosi ai rispettivi database per garantire che la documentazione rifletta accuratamente la struttura dati effettiva. Gli schemi OpenAPI generati vengono salvati come artifact e possono essere successivamente utilizzati per la validazione automatica, la generazione di client SDK o la pubblicazione su portali di documentazione API, garantendo che la documentazione tecnica sia sempre sincronizzata con il codice e accessibile agli sviluppatori e ai consumatori delle API.

Deploy with ArgocD

Gli stage `deploy:argo-tags` e `deploy:argo-tags-quality` gestiscono il deployment automatico dell'applicazione sugli ambienti Kubernetes attraverso ArgoCD[3]. Lo stage `deploy:argo-tags` si attiva esclusivamente per tag di release stable (formato major.minor.patch) ed effettua il deployment in ambiente di produzione, mentre `deploy:argo-tags-quality` gestisce le versioni pre-release (con suffissi alpha, beta, rc) deployandole nell'ambiente quality per test di integrazione e validazione. Entrambi gli stage utilizzano un'immagine Docker specializzata che contiene candymand, uno strumento wrapper sviluppato internamente che interagisce con ArgoCD per automatizzare il processo di deployment dichiarativo. Il tool riceve come parametri il namespace del progetto e la versione da deployare, quindi aggiorna automaticamente le configurazioni Git dei manifest Kubernetes, innescando il processo di GitOps gestito da ArgoCD che sincronizza lo stato desiderato con il cluster. Questo approccio garantisce deployment completamente tracciabili, rollback semplificati e piena aderenza ai principi di Infrastructure as Code, eliminando interventi manuali e riducendo il rischio di errori durante il rilascio.

Notify Release

Lo stage `notify:new-relic` si occupa di registrare automaticamente le nuove release dell'applicazione sulla piattaforma di Application Performance Monitoring *New Relic*, creando un marker di deployment che facilita la correlazione tra versioni del software e metriche di performance. Il job viene attivato per qualsiasi tag che rispetti il formato Semantic Versioning e utilizza l'API GraphQL di New Relic per identificare l'applicazione corretta tramite il suo GUID, distinguendo automaticamente tra l'istanza di produzione e quella di quality in base alla presenza o meno di suffissi pre-release nel tag. Una volta identificata l'entità monitorata, lo stage crea un evento di change tracking che registra la versione deployata, consentendo agli operatori e ai team di development di visualizzare nei dashboard di New Relic quando sono avvenuti i deployment e di correlare eventuali anomalie nelle performance o negli errori con specifiche versioni del software. Questo meccanismo di notifica automatica elimina la necessità di registrazione manuale dei deployment e fornisce una visibilità completa sul ciclo di vita dell'applicazione, supportando attività di troubleshooting, analisi di regressioni e valutazione dell'impatto delle release sulla stabilità del sistema.

[3]ArgoCD è uno strumento di GitOps per Kubernetes che mantiene automaticamente lo stato delle applicazioni nel cluster sincronizzato con quello dichiarato su Git.

3.2 Implementation of the Deployment Architecture on Kubernetes

L'architettura di deployment di SRM su Kubernetes è progettata per garantire scalabilità, resilienza e separazione delle responsabilità attraverso un'organizzazione basata su deployment distinti, ciascuno ottimizzato per funzioni specifiche del sistema.

La struttura ad alto livello dei file di deploy è la seguente:

```
namespace: servicerequestmanager
serviceAccount:
  create: TRUE
deployments:
  - name: static
    ...
  - name: backend
    ...
jobs:
  - name: sync-srp-catalog
    ...
  - name: sync-srp-sr-history
    ...
  - name: import-directory-data
    ...
```

Il sistema è isolato all'interno del namespace dedicato `servicerequestmanager`, garantendo la separazione logica delle risorse e facilitando la gestione dei permessi e delle network policies. La configurazione prevede la creazione di un service account dedicato, che permette ai pod di interagire con le API Kubernetes secondo il principio del *least privilege*, limitando i permessi solo a quelli strettamente necessari per il funzionamento del sistema.

3.2.1 Static Content Deployment

Il deployment `static` gestisce la distribuzione di contenuti statici attraverso un web server Nginx containerizzato, configurato per servire asset frontend quali JavaScript bundles, CSS, immagini e altri file statici. Questo deployment implementa una strategia di caching ottimizzata che riduce il carico sul backend e migliora significativamente i tempi di risposta per gli utenti finali.

```
  - name: static
    services:
      - name: static
```

```
4           port: 80
5       ingresses:
6         - name: static
7           public: False
8           type: http
9           host: servicerequestmanager.k8s.elmec.ad
10          path: /static
11          port: 80
12      containers:
13        - name: nginx
14          image: docker.elmec.com/.../ssl:production-latest
15          port: 80
16        - name: nginx-mtr-exp
17          image:
            ↪ docker.elmec.com/.../nginx-prometheus-exporter:0.8.0
18          port: 9113
19          args:
20            - "-nginx.scrape-uri"
21            - "http://127.0.0.1:8080/nginx_status"
22      replicaCount: 1
23      monitoring:
24        - name: nginx-metrics
25          port: 9113
26      imagePullSecrets:
27        - name: docker-registry-secret
```

Container Configuration

Il deployment si compone di due container in esecuzione sullo stesso pod. Il container principale `nginx` utilizza l'immagine `ssl:production-latest`, che include le configurazioni SSL/TLS e i certificati necessari per le comunicazioni sicure. Il server è configurato per ascoltare sulla porta 80 all'interno del cluster, mentre l'accesso esterno avviene attraverso il sistema di ingress che gestisce la terminazione SSL. Parallelamente, il container `nginx-mtr-exp` esegue il *Nginx Prometheus Exporter*, uno strumento specializzato che espone metriche operative del web server in formato compatibile con Prometheus. Questo sidecar container si connette all'endpoint `nginx_status` esposto da Nginx sulla porta 8080 e traduce le statistiche native in metriche Prometheus accessibili sulla porta 9113, permettendo al sistema di monitoraggio di raccogliere dati su throughput, latenza, errori e utilizzo delle connessioni.

Service and Ingress Configuration

Il servizio Kubernetes `static` espone il deployment sulla porta 80, fornendo un punto di accesso stabile indipendente dal lifecycle dei pod sottostanti. L'ingress associato è configurato come interno (non pubblico) e instrada le

richieste provenienti dall'host `servicerequestmanager.k8s.elmec.ad` con path `/static` verso il servizio, implementando un virtual hosting che permette di segregare il traffico statico da quello API.

Monitoring Integration

Il deployment integra nativamente il monitoraggio attraverso la definizione di un ServiceMonitor che istruisce Prometheus a scrapare le metriche esposte sulla porta 9113. Questa configurazione permette la raccolta automatica di telemetria operativa senza necessità di configurazioni manuali aggiuntive, garantendo visibilità continua sullo stato del servizio di contenuti statici.

3.2.2 Backend Application Deployment

Il deployment `backend` costituisce il core dell'architettura e gestisce l'intera logica applicativa attraverso un'organizzazione multi-container che separa le responsabilità tra API server e worker asincroni.

```
- name: backend
  vault:
    name: 'secrets'
  services:
    - name: api
      port: 5000
      mirror: TRUE
    - name: api-ext
      port: 5000
      mirror: True
  ingresses:
    - name: api
      public: False
      type: http
      host: servicerequestmanager.k8s.elmec.ad
      path: /
      port: 5000
      middlewares:
        - name: cors
          spec:
            headers:
              accessControlAllowMethods:
                - "GET"
                - "OPTIONS"
                - "POST"
                - "PUT"
                - "DELETE"
                - "PATCH"
```

```
            accessControlAllowOriginList:
              - "*"
            accessControlMaxAge: 100
            accessControlAllowHeaders: ["*"]
    - name: api-ext
      public: True
      type: http
      host: api.elmec.com
      path: /servicerequestmanager/
      port: 5000
      middlewares:
        - name: strip
          spec:
            replacePathRegex:
              regex: ^/servicerequestmanager/(.*)
              replacement: /$1
        - name: cors
          spec:
            headers:
              accessControlAllowMethods:
                - "GET"
                - "OPTIONS"
                - "POST"
                - "PUT"
                - "DELETE"
                - "PATCH"
              accessControlAllowOriginList:
                - "*"
              accessControlMaxAge: 100
              accessControlAllowHeaders: [ "*" ]
  containers:
    - name: api
      ...
    - name: celery
      ...
  replicaCount: 2
  monitoring:
    labels:
      port: "5000"
      host: "localhost"
    subscriptions:
      - django
    type: PodAndService
    podmonitors:
      - name: django-hc
        port: api
```

```
75          path: /ht/
76          healthCheck: True
77    imagePullSecrets:
78      - name: docker-registry-secret
```

Vault Integration for Secrets Management

Il deployment è configurato per integrare HashiCorp Vault attraverso l'injection automatica di secrets nel filesystem del pod. Il Vault Agent injector monta dinamicamente i segreti (credenziali database, chiavi API, password) nel path `/vault/secrets/secrets`, rendendo disponibili le informazioni sensibili ai container senza esporle nelle variabili d'ambiente o nei ConfigMap. Questo approccio implementa il pattern di secrets management secondo le best practice di sicurezza Kubernetes, garantendo rotazione automatica delle credenziali e audit trail completo degli accessi.

API Container Configuration

Il container `api` esegue l'applicazione Django attraverso il server Gunicorn, configurato per gestire richieste HTTP sulla porta 5000. La configurazione delle risorse implementa un modello di *Quality of Service Burstable*, con request di 400m CPU e 1000Mi di memoria e limit rispettivamente a 800m CPU e 1200Mi di memoria. Questa configurazione permette al container di scalare temporaneamente oltre le risorse richieste durante picchi di carico, pur garantendo un baseline minimo garantito dallo scheduler Kubernetes.

```
containers:
  - name: api
    image: "docker.elmec.com/innovation/servicerequestmanager:
    ↪ PROJECT_VERSION"
    port: 5000
    cpuRequest: 400m
    cpuLimit: 800m
    memoryRequest: 1000Mi
    memoryLimit: 1200Mi
    envs:
      - name: ENVIRONMENT
        value: "PRODUCTION"
      - name: DJANGO_DB_HOST
        value: 'servicerequestmanager-db-production-pooler-rw.
        ↪ autom-servicerequestmanager-production.svc.cluster
        ↪ .local'
      - name: DJANGO_DB_PORT
        value: '5432'
      ...
```

Le variabili d'ambiente definiscono la configurazione operativa dell'applicazione, includendo parametri di connessione al database PostgreSQL attraverso il pooler PgBouncer, configurazione del caching distribuito tramite Memcached, integrazione con Redis per la gestione delle code Celery, e endpoint di servizi esterni per autenticazione OIDC e integrazioni con altri sistemi aziendali. La configurazione include inoltre parametri specifici per l'integrazione con il sistema legacy SRP, definendo connessioni al database SQL Server (`elmec-cmi-mssql.elmec.ad/DbSRP`) e date di sincronizzazione per l'importazione dello storico delle service request. Il sistema di storage S3-compatible è configurato attraverso l'endpoint Ceph RGW (`rgw.elmec.com`) per la gestione dei log di esecuzione e import / export delle anagrafiche dei clienti.

Celery Worker Container

```
- name: celery
  image: "docker.elmec.com/innovation/servicerequestmanager:PR
  ↪ OJECT_VERSION"
  memoryLimit: 1200Mi
  memoryRequest: 1000Mi
  cpuRequest: 400m
  cpuLimit: 800m
  command: "/bin/sh"
  args:
    - -c
    - . /vault/secrets/secrets;cd servicerequestmanager;
    ↪ celery -A servicerequestmanager worker --loglevel=info
    ↪ --pool threads
  envs:
    - name: CELERY_QUEUE
      value: "servicerequestmanager_production"
    - name: REDIS_HOST
      value: "autom-servicerequestmanager-redis.autom-servicer
      ↪ equestmanager-production.svc.cluster.local"
    ...
```

Il container `celery` esegue worker Celery dedicati all'elaborazione asincrona di task in background legati al flusso di esecuzione delle service requests, utilizzando un thread pool per gestire operazioni concorrenti senza blocking. Il comando di avvio personalizzato inizializza l'ambiente caricando i secrets da Vault prima di avviare il worker con logging configurato a livello INFO. I worker sono configurati per consumare dalla coda dedicata `servicerequestmanag_production` su Redis, garantendo isolamento rispetto ad altri sistemi che condividono la stessa infrastruttura di messaging. L'allocation di risorse è identica a quella del container API, riflettendo requisiti computazionali simili.

Service Exposure and Load Balancing

Il deployment espone due servizi distinti per gestire traffico interno ed esterno.

Il servizio `api` opera internamente al cluster sulla porta `5000`, configurato con mirror mode attivo per permettere il traffic mirroring a scopi di testing o debugging. L'ingress associato è configurato come interno e instrada richieste provenienti dall'host `servicerequestmanager.k8s.elmec.ad` verso il backend, implementando middleware CORS che permette richieste cross-origin con metodi HTTP completi (`GET`, `POST`, `PUT`, `DELETE`, `PATCH`) e headers arbitrari.

Il servizio `api-ext` fornisce invece accesso pubblico attraverso l'host `api.elmec.com` con path prefix `/servicerequestmanager/`. La configurazione dell'ingress include un middleware di path rewriting che rimuove il prefix prima di inoltrare le richieste al backend, permettendo all'applicazione di operare con routing paths semplificati. Anche questo ingress implementa policy CORS permissive per supportare integrazioni da applicazioni web di terze parti.

Availability and Scaling

Il deployment è configurato con `replicaCount: 2`, garantendo ridondanza e distribuzione del carico su almeno due pod distribuiti su nodi differenti del cluster. Questa configurazione implementa alta disponibilità di base, permettendo rolling updates senza downtime e tolleranza ai failure di singoli pod o nodi. Lo scheduler Kubernetes distribuisce automaticamente i replica su nodi differenti quando possibile, massimizzando la resilienza.

Health Checking and Monitoring

Il sistema di monitoraggio è configurato attraverso PodMonitor che combina metriche applicative e health check. Il PodMonitor `django-hc` esegue check periodici sull'endpoint `/ht/` del servizio API, verificando che l'applicazione sia responsiva e correttamente configurata.

Il deployment è inoltre taggato con subscription django che attiva dashboard e alert predefiniti per applicazioni Django, fornendo visibilità immediata su metriche quali request rate, latency distribution, error rate e database connection pool utilization.

3.2.3 Scheduled Jobs for Data Synchronization

L'architettura include CronJob Kubernetes per l'esecuzione di task ricorrenti che mantengono allineati i dati tra ServiceRequestManager e il sistema legacy

SRP, in modo che i due sistemi, almeno inizialmente, possano co-esistere e restare allineati durante questo periodo.

```yaml
jobs:
  - name: sync-srp-catalog
    vault:
      name: 'secrets'
    schedule: "0 1 * * *"
    image: docker.elmec.com/innovation/servicerequestmanager:P
     ↪  ROJECT_VERSION
    memoryLimit: 700Mi
    memoryRequest: 400Mi
    cpuRequest: 400m
    cpuLimit: 800m
    command: "manage.sh"
    args:
      - "sync_srp_catalog"
    envs:
      ...
    imagePullSecrets:
      - name: docker-registry-secret
  - name: sync-srp-sr-history
    vault:
      name: 'secrets'
    schedule: "0 2 * * *"
    image: docker.elmec.com/innovation/servicerequestmanager:P
     ↪  ROJECT_VERSION
    memoryLimit: 700Mi
    memoryRequest: 400Mi
    cpuRequest: 400m
    cpuLimit: 600m
    command: "manage.sh"
    args:
      - "sync_srp_service_requests"
    envs:
      ...
    imagePullSecrets:
      - name: docker-registry-secret
```

SRP Catalog Synchronization

Il job `sync-srp-catalog` è schedulato per l'esecuzione quotidiana all'1:00 AM attraverso la cron expression `0 1 * * *`[4]. Questo job esegue il management command Django `sync_srp_catalog` che interroga il database di SRP per importare le configurazioni del catalogo. L'esecuzione è configurata con risorse più contenute rispetto ai deployment permanenti (400Mi-700Mi memoria, 400m-800m CPU), riflettendo la natura batch del workload che può tollerare latenze maggiori.

Service Request History Synchronization

Il job `sync-srp-sr-history` è schedulato per l'esecuzione alle 2:00 AM, un'ora dopo la sincronizzazione del catalogo per garantire che le configurazioni aggiornate siano già disponibili. Questo job esegue il comando `sync_srp_service_requests` che importa lo storico delle service request create nel sistema legacy a partire dalla data definita in `SRP_SR_SYNC_START_DATE`, mantenendo allineata la vista unificata delle richieste attraverso i due sistemi durante il periodo di transizione.

Entrambi i CronJob condividono la medesima configurazione di environment variables e secrets injection del deployment backend, garantendo consistenza nell'accesso a database e servizi esterni.

L'execution model basato su Job Kubernetes garantisce retry automatici in caso di failure e retention dello storico delle esecuzioni per debugging e auditing.

---

[4]La sintassi *cron* è un formato compatto utilizzato per programmare l'esecuzione periodica di processi o job nei sistemi Unix-like [5]. Si compone di cinque campi numerici separati da spazi (minuto, ora, giorno del mese, mese, giorno della settimana) che indicano quando un comando deve essere eseguito. Ogni campo può contenere valori singoli, intervalli (es. 1-5), liste (es. 1,3,7) o caratteri speciali come `*`, che significa "qualsiasi valore", e `/`, che indica una frequenza (ad esempio `*/5` per ogni cinque unità di tempo) [5]. Una stringa come `0 2 * * *` indica quindi l'esecuzione di un job tutti i giorni alle 02:00. Questa sintassi, pur essendo minimale, permette di descrivere in modo preciso e flessibile praticamente qualsiasi esigenza di schedulazione ricorrente.

3.3 Backend Service Implementation

3.3.1 Technological Overview

3.3.2 Catalog Management

3.3.3 Message Queue System for Asynchronous Execution Flow

3.3.4 Implementation of Customers' Master Data Import

3.3.5 Logging and Monitoring

3.4 Frontend Application Implementation

3.4.1 Technological Overview

3.4.2 Service Request Creation Platform for End Users

3.4.3 Administrative Configuration Platform for System Administrators

Chapter 4

Migrations

- Planning and execution of a full migration of databases and other company services / platforms that use SRP;
- Strategies to minimize downtime and ensure data integrity during the migration process
- Strategies to ensure backward compatibility during the transition period
- Strategies to ensure a smooth co-existence of the two systems during the transition period

Chapter 5

Reccomendation system through RAG and Fine-tuned LLMs

- Integration of Retrieval-Augmented Generation (RAG) techniques
- Fine-tuning of Large Language Models (LLMs)
- Implementation of a recommendation system for service requests

Conclusions

- Reflections on the internship experience and acquired skills
- Considerations on the future of the project and potential developments

Bibliography

[1] *Active Directory Organizational Unit (OU)*. URL: https://blog.netwrix.com/2024/02/19/active-directory-organizational-unit-ou/ (cit. on p. 23).

[2] Mashel Albarak and Rami Bahsoon. *Prioritizing Technical Debt in Database Normalization Using Portfolio Theory and Data Quality Metrics*. 2018. arXiv: 1801.06989 [cs.SE]. URL: https://arxiv.org/abs/1801.06989 (cit. on p. 16).

[3] Charles Anderson. "Docker [Software engineering]". In: *IEEE Software* 32.3 (2015), pp. 102–c3. DOI: 10.1109/MS.2015.62 (cit. on p. 28).

[4] *Ansible Documentation*. URL: https://docs.ansible.com/ansible/latest/dev_guide/overview_architecture.html (cit. on pp. 33, 34).

[5] *Automation with Cron job on Centos 8*. URL: https://comtronic.com.au/automation-with-cron-job-on-centos-8/ (cit. on p. 54).

[6] Michael R. Blaha. "Referential Integrity Is Important For Databases". In: 2005. URL: https://api.semanticscholar.org/CorpusID:6831872 (cit. on p. 16).

[7] *Cryptsetup and LUKS - open-source disk encryption*. URL: https://gitlab.com/cryptsetup/cryptsetup (cit. on p. 11).

[8] Lorenzo De Lauretis. "From Monolithic Architecture to Microservices Architecture". In: *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 2019, pp. 93–96. DOI: 10.1109/ISSREW.2019.00050 (cit. on p. 24).

[9] *DMI Infrastructure*. Internal repository, not publicly accessible. URL: https://git.elmec.com/publicautomation/dmi-infrastructure.git (cit. on pp. 8, 10, 11).

[10] Roy T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. University of California, Irvine, 2000 (cit. on p. 26).

[11] Paul J. Deitel Harvey M. Deitel. *C++. Fondamenti di programmazione*. Apogeo, 2005 (cit. on p. 16).

[12] *HDA - Help Desk Advanced*. URL: https://pat.eu/en/products/helpdeskadvanced/ (cit. on p. 27).

[13] *K3s - Lightweight Kubernetes*. URL: https://docs.k3s.io/ (cit. on p. 11).

[14] *Kubernetes: Production-Grade Container Orchestration*. URL: https://kubernetes.io/docs/concepts/overview/ (cit. on pp. 27, 29).

[15] Severi Peltonen, Luca Mezzalira, and Davide Taibi. *Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review*.

2021. arXiv: 2007.00293 [cs.SE]. URL: https://arxiv.org/abs/2007.00293 (cit. on pp. 12, 24).

[16] *Provision Prime*. Internal repository, not publicly accessible. URL: https://git.elmec.com/innovation/development/provisionprime.git (cit. on pp. 23, 24, 27).

[17] *Refactoring Guru - Strategy Pattern*. URL: https://refactoring.guru/design-patterns/strategy (cit. on p. 34).

[18] David K. Rensin. *Kubernetes - Scheduling the Future at Cloud Scale*. 2015, All. URL: http://www.oreilly.com/webops-perf/free/kubernetes.csp (cit. on pp. 29, 30).

[19] Elmec Informatica S.p.A. *Azioni Gestione SRP 4.0*. Internal document, not publicly available., 2016 (cit. on pp. 3, 5, 6, 26).

[20] Elmec Informatica S.p.A. *Requirements definition meeting for the new system*. Internal meeting. Held with the stakeholders and developers of the old system to gather requirements for the new system. Apr. 2025 (cit. on pp. 8, 16, 19–24).

[21] Elmec Informatica S.p.A. *Service Request Portal*. Internal document, not publicly available., 2016 (cit. on pp. 8, 9, 22).

[22] *Semgrep App Security Platform | AI-Assisted SAST, SCA and Secrets Detection*. URL: https://semgrep.dev/ (cit. on p. 41).

[23] *SQL Server Integration Services*. URL: https://learn.microsoft.com/en-us/sql/integration-services/sql-server-integration-services (cit. on p. 14).

[24] *SRP Internal Worker Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srpinternalworker_windowsservice.git (cit. on p. 10).

[25] *SRP Web Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WebService.git (cit. on p. 9).

[26] *SRP Web Service REST*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srp_webservicerest.git (cit. on pp. 9, 19, 22, 24).

[27] *SRP Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WindowsService.git (cit. on pp. 7, 10, 16, 22–24).

[28] Michael Stonebraker, Lawrence Rowe, and Michael Hirohama. "The Implementation Of Postgres". In: *Knowledge and Data Engineering, IEEE Transactions on* 2 (Apr. 1990), pp. 125–142. DOI: 10.1109/69.50912 (cit. on p. 35).

[29] *Stored Procedures (Database Engine)*. URL: https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine (cit. on p. 14).

[30] Jan Tretmans and Louis Verhaard. "A Queue Model Relating Synchronous and Asynchronous Communication". In: *Protocol Specification, Testing and Verification, XII*. Ed. by R.J. LINN and M.Ü. UYAR. IFIP

Transactions C: Communication Systems. Amsterdam: Elsevier, 1992, pp. 131–145. ISBN: 978-0-444-89874-6. DOI: https://doi.org/10.1016/B978-0-444-89874-6.50015-5. URL: https://www.sciencedirect.com/science/article/pii/B9780444898746500155 (cit. on p. 27).

[31] *Trivy - The all-in-one open source security scanner*. URL: https://trivy.dev/ (cit. on p. 40).

[32] *TypeScript: JavaScript with Syntax for Types*. URL: https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html (cit. on p. 30).

[33] Brecht Van de Vyvere, Pieter Colpaert, and Ruben Verborgh. "Comparing a Polling and Push-Based Approach for Live Open Data Interfaces". In: *Web Engineering*. Ed. by Maria Bielikova, Tommi Mikkonen, and Cesare Pautasso. Cham: Springer International Publishing, 2020, pp. 87–101. ISBN: 978-3-030-50578-3 (cit. on p. 22).

[34] *Vue.js v3 Documentation*. URL: https://vuejs.org/guide/introduction (cit. on pp. 30, 31).

[35] *What is a Container?* URL: https://www.docker.com/resources/what-container/ (cit. on p. 29).

[36] *What is referential integrity and what does it look like in practice?* URL: https://www.y42.com/blog/referential-integrity (cit. on p. 16).